Tools for Numerical Computing

Brian Howell

INDEFINITELY IN PROGRESS

January 12, 2026

Introduction:

I had the fortunate opportunity to pursue my graduate studies at UC Berkeley. Throughout my time, I was exposed to a handful of useful tools from applied mathematics, machine learning, optimization, and high performance computing. This document is an attempt to transcribe and compile my notes from a 5-year period into a reference guide for myself.

Disclaimer:

The intended audience was myself. That being said, I have shared sections of this document with colleagues and friends who claimed it was helpful in their studies. Maybe it will be helpful to you.

This is a live document that I intend to continue to update, which means there are likely to be some errors. If you find mistakes and feel like telling me, I will be grateful and happy to hear from you, even for the most trivial of errors. You can reach me by email at bhowell@berkeley.edu.

Contents

1 Recommended Readings

1.1 Mathematics

Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems

This book introduces finite difference methods for both ordinary differential equations (ODEs) and partial differential equations (PDEs) and discusses the similarities and differences between algorithm design and stability analysis for different types of equations. A unified view of stability theory for ODEs and PDEs is presented, and the interplay between ODE and PDE analysis is stressed. The text emphasizes standard classical methods, but several newer approaches also are introduced and are described in the context of simple motivating examples. Exercises and student projects are available on the book's webpage, along with Matlab mfiles for implementing methods. Readers will gain an understanding of the essential ideas that underlie the development, analysis, and practical use of finite difference methods as well as the key concepts of stability theory, their relation to one another, and their practical implications. The author provides a foundation from which students can approach more advanced topics.

A Finite Element Primer

The approach used herein is to introduce the basic concepts first in one dimension, then move on to three dimensions. A relatively informal style is adopted in this guide. These notes are intended as a "starting point," which can be later augmented by the large array of rigorous, detailed books in the area of Finite Element analysis. Through teaching finite element classes for a number of years at UC Berkeley, my experience has been that the fundamental mathematics, such as vector calculus, linear algebra, and basic mechanics, exemplified by linearized elasticity, cause conceptual problems that impede the understanding of the Finite Element Method. Thus, appendices on these topics have been included. Finally, I am certain that, despite painstaking efforts, there remain errors of one sort or another. Therefore, I would be grateful if readers who find such flaws would contact me at zohdi@berkeley.edu.

1.2 Optimization

Optimization Models:

Emphasizing practical understanding over the technicalities of specific algorithms, this elegant textbook is an accessible introduction to the field of optimization, focusing on powerful and reliable convex optimization techniques. Students and practitioners will learn how to recognize, simplify, model and solve optimization problems - and apply these principles to their own projects. A clear and self-contained introduction to linear algebra demonstrates core mathematical concepts in a way that is easy to follow, and helps students to understand their practical relevance. Requiring only a basic understanding of geometry, calculus, probability and statistics, and striking a careful balance between accessibility and rigor, it enables students to quickly understand the material, without being overwhelmed by complex mathematics. Accompanied by numerous end-of-chapter problems, an online solutions manual for instructors, and relevant examples from diverse fields including engineering, data science, economics, finance, and management, this is the perfect introduction to optimization for undergraduate and graduate students.

Convex Optimization

Convex optimization problems arise frequently in many different fields. A comprehensive introduction to the subject, this book shows in detail how such problems can be solved numerically with great efficiency. The focus is on recognizing convex optimization problems and then finding the most appropriate technique for solving them. The text contains many worked examples and homework exercises and will appeal to students, researchers and practitioners in fields such as engineering, computer science, mathematics, statistics, finance, and economics.

Algorithms for Optimization

This book offers a comprehensive introduction to optimization with a focus on practical algorithms. The book approaches optimization from an engineering perspective, where the objective is to design a system that optimizes a set of metrics subject to constraints. Readers will learn about computational approaches for a range of challenges, including searching high-dimensional spaces, handling problems where there are multiple competing objectives, and accommodating uncertainty in the metrics. Figures, examples, and exercises convey the intuition behind the mathematical approaches. The text provides concrete implementations in the Julia programming language.

1.3 Machine Learning and Data Science

Gaussian Processes for Machine Learning

Gaussian processes (GPs) provide a principled, practical, probabilistic approach to learning in kernel machines. GPs have received increased attention in the machine-learning community over the past decade, and this book provides a long-needed systematic and unified treatment of theoretical and practical aspects of GPs in machine

learning. The treatment is comprehensive and self-contained, targeted at researchers and students in machine learning and applied statistics. The book deals with the supervised-learning problem for both regression and classification, and includes detailed algorithms. A wide variety of covariance (kernel) functions are presented and their properties discussed. Model selection is discussed both from a Bayesian and a classical perspective. Many connections to other well-known techniques from machine learning and statistics are discussed, including support-vector machines, neural networks, splines, regularization networks, relevance vector machines and others. Theoretical issues including learning curves and the PAC-Bayesian framework are treated, and several approximation methods for learning with large datasets are discussed. The book contains illustrative examples and exercises, and code and datasets are available on the Web. Appendixes provide mathematical background and a discussion of Gaussian Markov processes.

Decision Making Under Uncertainty Theory and Application

Many important problems involve decision making under uncertainty—that is, choosing actions based on often imperfect observations, with unknown outcomes. Designers of automated decision support systems must take into account the various sources of uncertainty while balancing the multiple objectives of the system. This book provides an introduction to the challenges of decision making under uncertainty from a computational perspective. It presents both the theory behind decision making models and algorithms and a collection of example applications that range from speech recognition to aircraft collision avoidance.

2 Maths

2.1 Calculus

The following section on calculus was written by *rdhuff@berkeley.edu*

2.1.1 Fundamentals

2.1.2 Einstein summation notation

Einstein summation notation is a shorthand way of writing mathematical expressions that involve repeated summations or products over indices. It is widely used in fields such as physics and engineering to write complex equations in a more concise and elegant form.

In Einstein summation notation, repeated indices are automatically summed over, unless they appear once in a subscript and once in a superscript. This convention is known as the Einstein summation convention. The indices are typically Greek letters for time indices, and Latin letters for spatial indices.

In this notation, scalars, vectors, and matrices are represented as follows:

- Scalars are represented by a single symbol, such as a or f.
- Vectors are represented by lowercase Latin letters with indices, such as v_i or u_j. The indices can take on values of 1, 2, ..., n, where n is the dimension of the vector.
- Matrices are represented by uppercase Latin letters with two indices, such as A_{ij} or B_{kl}. The first index refers to the row number and the second index refers to the column number.

Using this notation, we can write linear algebra equations more compactly and with less clutter. For example, the dot product of two vectors $\mathbf{u}$ and $\mathbf{v}$ can be written as:

$$\mathbf{u} \cdot \mathbf{v} = u_i v_i \tag{1}$$

Similarly, the matrix-vector product $\mathbf{y} = A\mathbf{x}$ can be written as:

$$y_i = A_{ij} x_j \tag{2}$$

Einstein summation notation can also be used for expressing more complex operations such as matrix multiplication, transpose, and inverse.

Kronecker Delta:

$$\delta_{ij} = \mathbf{e}_i \cdot \mathbf{e}_j = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \tag{3}$$

Dot Product:

$$\mathbf{u} \cdot \mathbf{v} = (u_i \mathbf{e}_i) \cdot (v_j \mathbf{e}_j) = u_i v_j \mathbf{e}_i \cdot \mathbf{e}_j = u_i v_j \delta_{ij} = u_i v_i \tag{4}$$

Alternating (Epsilon or Levi-Civita) Symbol:

$$\varepsilon_{ijk} = \begin{cases} 1 & i,j,k \text{ is an even permutation} \\ 0 & i = j \text{ or } i = k \text{ or } j = k \\ -1 & i,j,k \text{ is an odd permutation} \end{cases} \tag{5}$$

Cross Product:

$$u \times v = \varepsilon_{ijk} u_j v_k e_i \tag{6}$$

$$(u \times v) i = \varepsilon i j k u_j v_k \tag{7}$$

Relationship between the Alternating Symbol and the Kronecker Delta:

$$\varepsilon_{ijk}\varepsilon_{ilm} = \delta_{jl}\delta_{km} - \delta_{jm}\delta_{kl} \tag{8}$$

Tensor (Dyadic) Product:

$$(a \otimes b)u = a(b \cdot u) \tag{9}$$
$$(a \otimes b)_{ij} = a_i b_j \tag{10}$$
$$(a \otimes b)(a \otimes b) = a(b \cdot a) \otimes b \tag{11}$$
$$(a \otimes b) = (a_i e_i) \otimes (b_j e_j) = a_i b_j e_i \times e_j \tag{12}$$
$$I = \delta_{ij} e_i \otimes e_j = e_i \otimes e_i \tag{13}$$

Gradient of Scalar Field: The gradient of a scalar field is a vector that points in the direction of the greatest increase of the function at a given point. It is denoted by the symbol ∇.

In Cartesian coordinates, the gradient of a scalar function $f(x, y, z)$ is defined as:

$$\nabla f = \frac{\partial f}{\partial x}\mathbf{i} + \frac{\partial f}{\partial y}\mathbf{j} + \frac{\partial f}{\partial z}\mathbf{k}, \tag{14}$$

where $\mathbf{i}, \mathbf{j}$, and $\mathbf{k}$ are the unit vectors along the x, y, and z axes, respectively.

More generally, the gradient of a scalar field $f(\mathbf{x})$ in n-dimensional space can be written as:

$$\nabla f = \frac{\partial f}{\partial x_1}\mathbf{e_1} + \frac{\partial f}{\partial x_2}\mathbf{e_2} + \cdots + \frac{\partial f}{\partial x_n}\mathbf{e_n}, \tag{15}$$

where $\mathbf{e_1}, \mathbf{e_2}, \ldots, \mathbf{e_n}$ are the unit vectors along the n coordinate axes.

By convention, if we wish to represent the gradient of a scalar function $f(\mathbf{x})$ as a vector in direct notation, we write it as a column vector:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \tag{16}$$

This makes sense because the gradient is a vector quantity, and vectors are typically represented as column vectors.

Example. Let's consider the scalar function $f(\mathbf{x}) = \mathbf{x}^T A\mathbf{x}$, where $A \in \mathbb{R}^{n \times n}$. The gradient of this function with respect to $\mathbf{x}$ is:

$$\nabla_{\mathbf{x}} f = \frac{\partial f}{\partial x_i} e_i \tag{17}$$
$$= \frac{\partial}{\partial x_i}(x_j a_{jk} x_k) e_i \tag{18}$$
$$= (2a_{ij}x_j)e_i + (a_{jk}x_k)\frac{\partial x_j}{\partial x_i} e_i \tag{19}$$
$$= (A + A^T)_{ij} x_j e_i, \tag{20}$$

where we used the product rule for partial differentiation and the fact that A is symmetric. The expression $(A + A^T)_{ij}$ represents the (i, j)-th entry of the matrix $A + A^T$.

Gradient of Vector Field:

$$\nabla v = \frac{\partial v_i}{\partial x_j} e_i \otimes e_j \tag{21}$$

2.2 Differential Equations

2.2.1 Parabolic Partial Differential Equations

Parabolic partial differential equations (PDEs) play a fundamental role in modeling various phenomena, ranging *from black holes to Black-Scholes*. More specifically, the form of parabolic PDEs elegantly captures the dynamics of a large range processes such as heat and chemical diffusion, Schrödinger's equation, and option pricing in financial mathematics. One of the most commonly used numerical methods for solving such equations is the finite difference method (FDM). FDM is a simple, efficient, and robust technique that discretizes the PDE on a grid and approximates the derivatives using finite differences [**?**]. This method has been extensively studied and applied in various fields of science and engineering for over a century. In this section of the dissertation, I discuss the theoretical foundations and practical implementation of FDM for parabolic PDEs.

$$\underbrace{u_t}_{\text{time differential}} - \underbrace{\nabla_x \cdot (\kappa \nabla_x u))}_{\text{diffusion of quantity } u} = \underbrace{f_m}_{\text{source terms}} \tag{22}$$

The general form of the classic heat equation is shown in Equation (22) where u models temperature or chemical concentration at a given point in space and time, κ is the heat capacity or diffusion coefficient (possibly temporally and/or spatially dependent), and f_m, $\forall m = 1, ..., M$ nodes are the source term (also possibly temporally and/or spatially dependent). To solve the heat equation numerically, it is necessary to specify both initial conditions and boundary conditions. The initial conditions specify the temperature distribution within the domain at time zero, while the boundary conditions specify the behavior of the solution at the boundaries of the domain. Dirichlet boundary conditions prescribe the temperature on the boundary, while Neumann boundary conditions prescribe the flux through the boundary. These boundary conditions are required because they provide the necessary information to obtain a unique solution to the heat equation. Without boundary conditions, the solution would not be well-defined, and there would be an infinite number of possible solutions that satisfy the PDE. Therefore, the appropriate choice of boundary conditions is crucial for obtaining physically meaningful solutions to the heat equation.

Concretely, a well-defined parabolic PDE consists of:

1. PDE: $u_t - \nabla_x \cdot (\kappa \nabla_x u) = f, \quad x \in \Omega, \quad t > 0$
2. Initial condition: $u(x, t = 0) = \eta(x), \quad x \in \Omega$
3. Boundary conditions: $\Gamma = \partial\Omega$
   - Dirichlet boundary condition: $u = u_D$ on $\Gamma \times (0, T)$ which directly prescribes the temperature or chemical concentration.
   - Neumann boundary condition: $(\kappa \nabla_x u) \cdot \boldsymbol{n} = g_M$ on $\Gamma \times (0, T)$ which prescribes the heat or diffusion flux across the boundary.

2.2.2 Finite Difference Grid

For simple geometries, the finite difference grid provides a straight-forward approach to discretizing the domain into a finite number of points or nodes. Each node represents a discrete location where the solution is evaluated, and the solution at each node is approximated using a finite difference approximation (example in Figure 1).

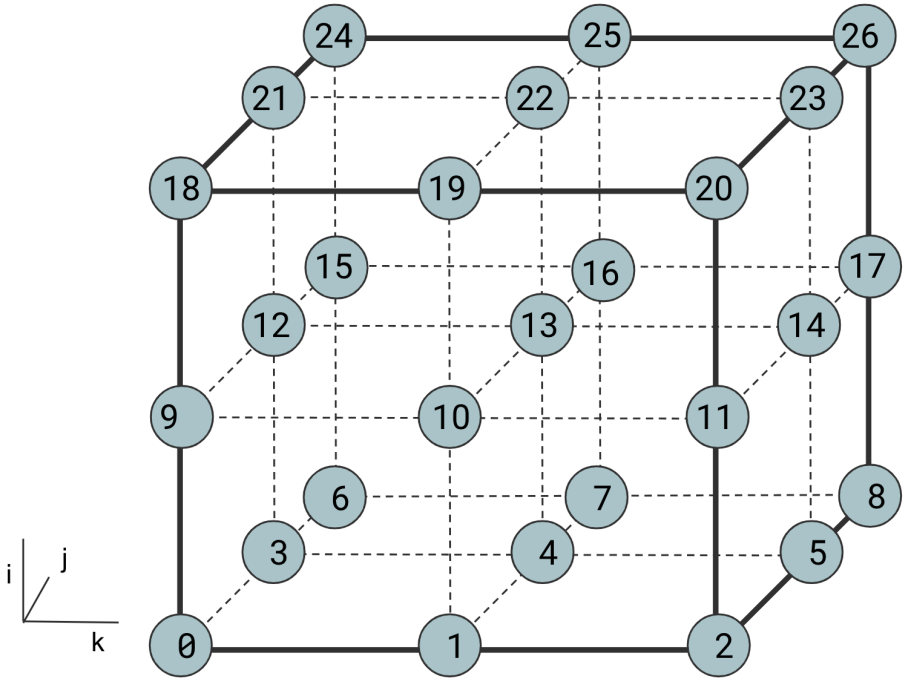


Figure 1: An example 3D finite difference grid where each node is equally spaced from neighboring nodes by some amount $\Delta x = \frac{L}{M-1}$, where L is the length of the domain across a single axis, and M are the total number of nodes along that same axis.

The accuracy of the FDM depends on the *size of the grid* and *the order of the finite difference approximation* used. In this way, the spatial domain of the PDE is transformed into a discrete set of equations that can be solved numerically using linear algebra techniques. The choice of grid size and finite difference approximation is critical in achieving a stable and accurate numerical solution.

2.2.3 Finite Difference Methods

Forward Difference in Time, Centered in Space (FTCS)

We seek to find an approximate solution $U_m^n \approx u(x_m, t_n)$ by approximating the diffusion term in Equation (22) which will enable us to "march forward in time", computing $U_m^{n+1}\ \forall\ m = 1, \ldots, M$ as a function of U_m^n at the previous time step.

FTCS seeks to find an approximate solution $U_m^n \approx u(x_m, t_n)$ by spatially discretizing the domain into a finite number of equally spaced grid points and approximates the spatial derivative at each point using a centered difference formula. The temporal derivative is approximated using a forward difference formula, and the solution at each grid point is updated at each time step based on the values at its neighboring points in space and the previous time step.

To obtain the FTCS approximation for $x \in \mathcal{R}^1$, we first consider a second-order center-difference approximation [**?**] of the Laplacian of the flux:

$$\nabla_x \cdot \boldsymbol{q} = \frac{\partial q}{\partial x} \approx \frac{q(x_1 + h/2) - q(x_1 - h/2)}{h} \tag{23}$$

where $\boldsymbol{q} = \kappa \nabla_x u$ and $h = \Delta x$. Assuming κ is constant in space, each partial flux can be further approximated by applying a successive first-order one-sided approximation:

$$q\big(x + h/2\big) \approx \kappa \frac{u(x+h) - u(x)}{h} \tag{24}$$

$$q\big(x - h/2\big) \approx \kappa \frac{u(x) - u(x-h)}{h} \tag{25}$$

yielding the centered-in-space approximation:

$$\frac{\partial}{\partial x}\left(\kappa\frac{\partial u}{\partial x}\right) = \kappa u_{xx} \approx \kappa\frac{u(x-h) - 2u(x) + u(x+h)}{h^2}. \tag{26}$$

Finally, the time derivative on the left-hand side of Equation (22) is approximated using a first-order one-sided approximation

$$U_m^{n+1} = U_m^n + \frac{k\kappa}{h^2}\left(U_{m-1}^n - 2U_m^n + U_{m+1}^n\right) + \frac{k}{h^2}f(U_m^n, t_n) \tag{27}$$

where $k = \Delta t$. This method is more commonly referred to as the *Forward Euler* time stepping scheme. The FTCS method is considered to be a first-order accurate in time, and second-order accurate in space. Since the right-hand side of Equation (27) is only dependent on U_m^n at the previous time step (all of which are known), this scheme is considered to be in an *explicit method* since U_m^{n+1} can be explicitly computed.

Trapezoidal Method

Like the FTCS method, the trapezoidal method also employs a second-order center-difference approximation for the spatial discretization of the diffusion term in Equation (22). However, rather than using an exclusive forward difference scheme, the trapezoidal method employs a weighted average of backward and forward differences in time parameterized by $\phi \in [0, 1]$:

$$\begin{aligned} U_m^{n+1} = \;& U_m^n \\ &+ \phi\frac{k}{h^2}\left(\kappa(U_{m-1}^n - 2U_m^n + U_{m+1}^n) + f(U_m^n, t_n)\right) \\ &+ (1-\phi)\frac{k}{h^2}\left(\kappa\underbrace{(U_{m-1}^{n+1} - 2U_m^{n+1} + U_{m+1}^{n+1}) + f(U_m^{n+1}, t_n)}_{U_m^{n+1}\ \textit{is unknown}}\right). \end{aligned} \tag{28}$$

When $\phi = 1/2$, the trapezoidal method reduces to the *Crank-Nicholson* method. The trapezoidal/Crank-Nicholson method is considered to be a second-order accurate in time, and second-order accurate in space. Since the right-hand side of Equation (27) is only dependent on both U_m^n at the previous time step (all of which are known) and U_m^{n+1} at the next time step (all of which are unknown), this scheme is considered to be in an *implicit method* since U_m^{n+1} must be implicitly computed.

Example: Crank-Nicholson for linear PDEs

Consider the simple parabolic PDE:

$$u_t = u_{xx}. \tag{29}$$

The Crank-Nicholson method discretizing Equation (29) yields:

$$U_m^{n+1} = U_m^n + \frac{k}{2h^2}\Big(U_{m-1}^n - 2U_m^n + U_{m+1}^n + U_{m-1}^{n+1} - 2U_m^{n+1} + U_{m+1}^{n+1}\Big). \tag{30}$$

In the case in which the PDE is linear, we can move all terms containing the unknown solutions U_m^{n+1} at the text time step to the left-hand side and leave all of the known solutions U_m^n at the previous time step on the right-hand side:

$$-\frac{k}{2h^2}U_{m-1}^{n+1} + (1+2\frac{k}{2h^2})U_m^{n+1} - \frac{k}{2h^2}U_{m+1}^{n+1} = \underbrace{\frac{k}{2h^2}U_{m-1}^n + (1-2\frac{k}{2h^2})U_m^n - \frac{k}{2h^2}U_{m+1}^n}_{known\ \boldsymbol{b}}. \tag{31}$$

Our known solutions at the previous and next time steps can be organized as vectors $\boldsymbol{U}_m^n, \boldsymbol{U}_m^{n+1} \in \mathcal{R}^{m_{nodes}}$ respectively. This means we can re-write the left-hand side of Equation (31) as system of M equations, and the right-hand side as a single column vector:

$$\underbrace{\begin{bmatrix} (1+2\frac{k}{2h^2}) & -\frac{k}{2h^2} & 0 & \dots & & 0 \\ -\frac{k}{2h^2} & (1+2\frac{k}{2h^2}) & -\frac{k}{2h^2} & \dots & & \vdots \\ 0 & \ddots & \ddots & \ddots & & \\ \vdots & & & -\frac{k}{2h^2} & (1+2\frac{k}{2h^2}) & -\frac{k}{2h^2} \\ 0 & & \dots & & -\frac{k}{2h^2} & (1+2\frac{k}{2h^2}) \end{bmatrix}}_{\boldsymbol{A}} \underbrace{\begin{bmatrix} U_1^{n+1} \\ U_2^{n+1} \\ \vdots \\ U_{M-1}^{n+1} \\ U_M^{n+1} \end{bmatrix}}_{\boldsymbol{x}} = \boldsymbol{b} \tag{32}$$

where

$$\boldsymbol{b} = \begin{bmatrix} \frac{k}{2h^2}\big(g_0(t_n) + g_0(t_{n+1})\big) + (1-\frac{k}{2h^2})U_1^n + \frac{k}{2h^2}U_2^n \\ \frac{k}{2h^2}U_1^n + (1-2\frac{k}{2h^2})U_2^n + \frac{k}{2h^2}U_3^n \\ \vdots \\ \frac{k}{2h^2}U_{M-2}^n + (1-2\frac{k}{2h^2})U_{M-1}^n + \frac{k}{2h^2}U_M^n \\ \frac{k}{2h^2}U_{M-1}^n + (1-2\frac{k}{2h^2}) + \frac{k}{2h^2}\big(g_1(t_n) + g_1(t_{n+1})\big) \end{bmatrix}$$

which reduces to solving $\boldsymbol{Ax} = \boldsymbol{b}$. Since $\boldsymbol{A}$ is tri-diagonal, this problem can be solved as efficiently as an explicit method.

2.2.4 Analysis of Numerical Methods

Local Truncation Error and Order of Accuracy

Numerical methods provide approximate solutions for ODEs/PDEs, and the accuracy of the solutions are dependent on the time discretization $k = \Delta t$ and spatial discretization $h = \Delta x$. After taking a single time step, some amount of error is introduced to the solution, also referred to as the local truncation error (LTE). This provides a method for analyzing the order of accuracy of a numerical method and how the quickly the error decreases as $h,\ k \to 0$. The LTE for the simple ODE $u_t = \lambda u + f(t)$ with an initial value $u(0) = u_0$ for the Backward Euler time discretization method is:

$$\tau_n = \frac{u(t_n) - u(t_{n-1})}{k} - \lambda u(t_n) - f(t_n), \quad \forall n = 1, ..., N \tag{33}$$

Order of accuracy is a measure of how well a finite difference method approximates a solution to a differential equation and describes how quickly the error between the approximate solution and the true solution decreases as $k,\ h \to 0$. Intuitively, a method will converge to the true solution at rate proportional to the order of accuracy. For example, a method with second order of accuracy in space will have an error that decreases proportional to the square of the grid spacing. Although we *expect* the approximate solution to approach the true solution such that the error of the method $||\tau(x,t)|| \to 0$, for convergence to the true solution to occur, the temporal and spatial discretization must satisfy a the two properties: consistency and stability.

Consistency

As both $h,\ k \to 0$, the we *expect* the approximate solution to approach the true solution such that the LTE of the method $||\boldsymbol{\tau}|| \to 0$. If this is indeed true, the discretization method is said to be *consistent*, meaning that the discrete equation approximates the same process as the underlying PDE as the temporal and spatial grids become finer at a rate that is at least as fast as $h,\ k \to 0$.

More concretely, *consistency* requires:

$$||\boldsymbol{\tau}||_\infty = \max_{n=1,\dots,N} |\tau_n| \ \to 0 \text{ as } \Delta t \to 0. \tag{34}$$

In the *LTE and order of accuracy for the Crank-Nicolson Method* example, the LTE is shown to be $\mathcal{O}(k^2 + h^2)$. A method is said to be *consistent* since the global order of accuracy will agree with the order of the LTE:

$$U_m^n - u(X,T) = \mathcal{O}(k^2 + h^2). \tag{35}$$

To determine whether a method is consistent, we need to analyze LTE. The following are two examples for deriving the LTE and order of accuracy for the *Forward in Time, Centered in Space* and *Crank-Nicolson* methods, and how the error corresponds to $k,\ h$.

Example: LTE and order of accuracy for FTCS method
Consider the simple parabolic PDE $u_t = u_{xx}$. The resulting error for a chosen time step k and for a chosen spatial step h for Equation (27) is defined as:

$$\tau(x,t) = \frac{U_m^{n+1} - U_m^n}{k} - \frac{\kappa}{h^2}\big(U_{m-1}^n - 2U_m^n + U_{m+1}^n\big). \tag{36}$$

To obtain the LTE, insert the true solution:

$$\tau(x,t) = \frac{u(x,t+k) - u(x,t)}{k} - \frac{\kappa}{h^2}\big(u(x-h,t) - 2u(x,t) + u(x+h,t)\big) \tag{37}$$

For the first term on the right-hand side, we care about the error relative to k, and for the second term we care about the error relative to h. If we make the assumption that u is smooth, we can expand each term around k, h respectively:
Assuming that u is sufficiently smooth and differentiable, each u term containing increment in k and h can be expanded in a Taylor series:

$$u(x,t+k) = u + ku_t + \frac{1}{2}k^2u_{tt} + \frac{1}{6}k^3u_{ttt} + \mathcal{O}(t^4) \tag{38}$$

$$u(x-h,t) = u - hu_x + \frac{1}{2}h^2u_{xx} - \frac{1}{6}h^3u_{xxx} + \mathcal{O}(h^4) \tag{39}$$

$$u(x+h,t) = u + hu_x + \frac{1}{2}h^2u_{xx} + \frac{1}{6}h^3u_{xxx} + \mathcal{O}(h^4). \tag{40}$$

Inserting each Taylor series expansion from Equation (38), Equation (39), and Equation (40) into Equation (37) yields:

$$\tau(x,t) = \Big(\underbrace{u_t}_{=\,u_{xx}} + \underbrace{\frac{1}{2}ku_{tt}}_{=\,u_{txx}=\,u_{xxxx}} + \mathcal{O}(k^2)\Big) - \Big(u_{xx} + \frac{1}{12}h^2u_{xxxx} + \mathcal{O}(h^3)\Big) \tag{41}$$

Recall that $u_t = u_{xx}$ and $u_{tt} = u_{xxxx}$ allowing for additional terms to simplify:

$$\tau(x,t) = \big(\frac{1}{2}k - \frac{1}{12}h^2\big)u_{xxxx} = \mathcal{O}(k + h^2).$$

This derivation shows that the error for the FTCS method is expected to increase linearly with k and to increase with the power of two with h. This is more commonly stated to be *first-order accurate in time, and second-order accurate in space.*

Example: LTE and order of accuracy for the Crank-Nicolson method
Once again, consider the simple parabolic PDE $u_t = u_{xx}$. The resulting error for a chosen time step k and for a chosen spatial step h is defined as:

$$\tau(x,t) = \frac{U_m^{n+1} - U_m^n}{k} + \frac{1}{2h^2}\left(U_{m-1}^n - 2U_m^n + U_{m+1}^n + U_{m-1}^{n+1} - 2U_m^{n+1} + U_{m+1}^{n+1}\right). \tag{42}$$

To obtain the LTE, insert the true solution:

$$\begin{aligned}\tau(x,t) \;=\; & \frac{u(x,t+k) - u(x,t)}{k} \\ & + \frac{1}{2h^2}\Big(u(x-h,t) - 2u(x,t) + u(x+h,t) \\ & \qquad + \; u(x-h,t+k) - 2u(x,t+k) + u(x+h,t+k)\Big).\end{aligned} \tag{43}$$

Once again assuming the true solution u is sufficiently smooth and differentiable, the Taylor series expansion for $u(x,t+k)$, $u(x-h,t)$ and $u(x+h,t)$ can be recycled from Equation (38), Equation (39), Equation (40). For $u(x-h,t+k)$ and $u(x+h,t+k)$, a multi-variable Taylor series expansion is required:

$$u(x-h,t+k) = u - hu_x + ku_t + \frac{1}{2}h^2u_{xx} - hku_{tx} + \frac{1}{2}k^2u_{tt} + \mathcal{O}(h^3,k^3,h^2k,hk^2) \tag{44}$$

$$u(x+h,t+k) = u + hu_x + ku_t + \frac{1}{2}h^2u_{xx} + hku_{tx} + \frac{1}{2}k^2u_{tt} + \mathcal{O}(h^3,k^3,h^2k,hk^2). \tag{45}$$

Inserting each Taylor series expansion from Equations 38-40 and Equations 44-45 into Equation (43) and a lot of tedious cancellation of terms yields:

$$\tau(x,t) = \mathcal{O}(k^2,h^2). \tag{46}$$

This derivation shows that the error for the Crank-Nicolson method is expected to increase with the power of two with k and to increase with the power of two with h. This is more commonly stated to be *second-order accurate in time, and second-order accurate in space.*

Stability

Small variations in the initial and boundary conditions may cause sensitive numerical methods to be unstable. These small LTEs may cause the approximate solution to diverge from the true solution over time since each error may compound on itself. For time-dependent ODEs, the concept of *absolute stability* allows us to analyze terms that recursively amplify or dampen a solution after a time step. The stability theory for time-dependent PDEs can be related to the stability theory for time-dependent ODEs through the method of lines (MOL) discretization of the PDE (see Figure 2). The MOL approach first discretizes in space alone, resulting in a system of ODEs that can be solved using methods for ODEs. This system of ODEs is known as a semi-discrete method since it is discretized in space but not yet in time.

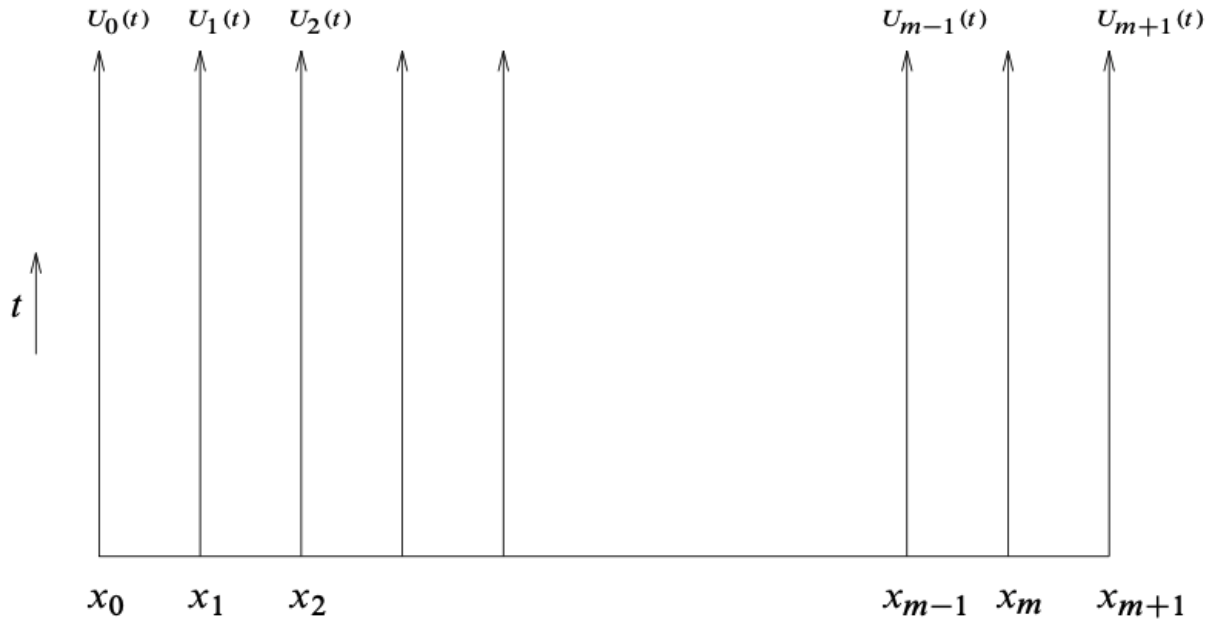


Figure 2: Method of lines interpretation. $U_i(t)$ is the solution along the line forward in time at the grid point x_i [**?**].

Once system of semi-discrete ODE's has been obtained from the MOL, we can apply our chosen time stepping scheme to obtain fully discrete method in which we can isolate the amplifying or dampening factor that affects stability. For ODE absolute stability, this factor is a scalar value (see *Absolute Stability* example below). However, once the MOL has been applied to a PDE, we obtain a system of equations that can be represented as matrices and vectors (if linear). This means the amplifying/dampening factor can be described by analyzing eigenvalues of matrices.

The following provides explicit examples for *Absolute stability*, *Stability Analysis of FTCS* and *Stability analysis of Crank-Nicolson*.

Example: Absolute stability of Forward Euler
The notion of *absolute stability* states that an ordinary differential equation (ODE) is absolutely stable under a numerical method if the numerical solution remains bounded as the step size or time step approaches zero, regardless of initial conditions. Consider the following classic ODE:

$$u_t = \lambda u(t) + g(t), \quad u(t_0) = \eta \tag{47}$$

and its discretization using Forward Euler time stepping

$$U^{n+1} = (1 + k\lambda)U^n + kg(t) \tag{48}$$

where the error

$$k\tau^n = u(t_{n+1}) - u(t_n) - k\big(\lambda u(t_n) + g(t_n)\big) \tag{49}$$

and its LTE:

$$\tau^n = \frac{1}{2}ku_{tt}(t_n) + \mathcal{O}(k^2). \tag{50}$$

To obtain the global error, subtract the following:

$$u(t_{n+1}) = (1 + k\lambda)u(t_n) + kg(t_n) + k\tau^n \tag{51}$$
$$U^{n+1} = (1 + k\lambda)U^n + kg(t_n) \tag{52}$$

to get

$$E^{n+1} = (1 + k\lambda)E^n - k\tau^n. \tag{53}$$

As the numerical method marches in time, the error is recursively scaled by $(1 + k\lambda)$. For $N + 1$ total time steps, the final global error is:

$$E^{N+1} = (1 + k\lambda)^N E^N - k\tau^N. \tag{54}$$

where the superscript N above E represents the second to last time step and the superscript N above $(1+k\lambda)$ obnoxiously represents a power. We expect the to see an exponential grown factor any k for which $|1+\lambda k| > 1$. Therefore we require $|1+k\lambda| \leq 1$ for the Forward Euler method to be absolutely stable. More generally, since λ maybe complex, the region of absolute stability is a disk of radius 1, centered at the point -1 (see Figure 3).

Example: Absolute stability of Trapezoidal
Once again, consider the following classic ODE:

$$u_t = \lambda u(t) + g(t), \quad u(t_0) = \eta \tag{55}$$

and its discretization using Trapezoidal time stepping

$$U^{n+1} = U^n + \frac{k}{2}\big(\lambda U^n + g(t)\big) + \frac{k}{2}\big(\lambda U^{n+1} + g(t_{n+1})\big) \tag{56}$$

and solving for U^{n+1} and inserting exact solution yields:

$$U^{n+1} = \frac{(1+\frac{k\lambda}{2})}{(1-\frac{k\lambda}{2})}U^n + \frac{\frac{k}{2}(g^n + g^{n+1})}{(1-\frac{k\lambda}{2})} \tag{57}$$

$$U(t_{n+1}) = \frac{(1+\frac{k\lambda}{2})}{(1-\frac{k\lambda}{2})}U(t_n) + \frac{\frac{k}{2}(g^n + g^{n+1})}{(1-\frac{k\lambda}{2})} + k\tau^n. \tag{58}$$

Subtracting the two and analyzing for the global error:

$$E^{N+1} = \rho^N E^N - k\tau^N, \quad \textit{where } \rho = \frac{(1+\frac{k\lambda}{2})}{(1-\frac{k\lambda}{2})}. \tag{59}$$

where the superscript N above E represents the second to last time step and the superscript N above ρ obnoxiously represents a power. For absolute stability, we require $|\rho| < 1$. By induction, it is simple to see that $|\rho| < 1 \; \forall \; \lambda k < 0$. This means that the trapezoidal method's stability region is the entire left-hand plane in Figure 3.

Example: Stability analysis of FTCS

As with the analysis of *absolute stability* for an ODE, analysis can be performed on the FTCS method by using first spatially discretizing in space using *Method of Lines* (MOL) to obtain a system of ODEs to interpret the method as a time integration methods of ODEs.
For the simple heat equation $u_t = u_{xx},\ u(t=0) = u_0$, the MOL for FTCS yields a semi-discrete ODE:

$$u_t = \frac{1}{h^2}(U^n_{m-1} - 2U^n_m + U^n_{m+1}), \quad m = 1, \dots, M. \tag{60}$$

After temporally discretizing, the boundary conditions can be explicitly encoded in the matrix form:

$$\boldsymbol{U}^{n+1} = (\boldsymbol{I} + k\boldsymbol{A})\boldsymbol{U}^n + k\boldsymbol{g} \tag{61}$$

where

$$\boldsymbol{A} = \frac{1}{h^2}\begin{bmatrix} -2 & 1 & & & & & \\ 1 & -2 & 1 & & & & \\ & \ddots & \ddots & \ddots & & & \\ & & & & 1 & -2 & 1 \\ & & & & & 1 & -2 \end{bmatrix}, \quad \boldsymbol{g} = \frac{1}{h^2}\begin{bmatrix} g_0(t) = U_0(t) \\ 0 \\ \vdots \\ 0 \\ g_1(t) = U_M(t) \end{bmatrix}. \tag{62}$$

As in the *absolute stability* example, we can compute the final global error:

$$\boldsymbol{E}^{N+1} = (\boldsymbol{I} + k\boldsymbol{A})^N \boldsymbol{E}^N - k\boldsymbol{\tau}^N \tag{63}$$

where the superscript N above $\boldsymbol{E}$ represents the second to last time step and the superscript N above $(\boldsymbol{I}+k\boldsymbol{A})$ obnoxiously represents a power. More importantly than the inconsistent notation are the eigenvalues of $\boldsymbol{A}$ which determine whether the global error converges or diverges. For the FTCS method to be stable, we require that $k\lambda_m \in \mathcal{S}$.
For this simple heat equation, the eigenvalues of $\boldsymbol{A}$ are:

$$\lambda_m = \frac{2}{h^2}\big(\cos(m\pi h) - 1\big), \quad \forall m = 1, \dots, M. \tag{64}$$

where $h = \frac{1}{m+1}$. The magnitude of $|\lambda_m|$ is largest when $m = M$ and resulting in the eigenvalue:

$$\lambda_M \approx \frac{2}{h^2}\big(cos(M\pi h) - 1\big) = \frac{-4}{h^2}. \tag{65}$$

Therefore we require that $-k\frac{4}{h^2} \in \mathcal{S}$, which for Forward Euler corresponds to $-2 \le -k\frac{4}{h^2} \le 0$, or $\frac{k}{h^2} \le \frac{1}{2}$.

Example: Stability analysis of Crank-Nicolson

As with the analysis of *absolute stability* for an ODE, analysis can be performed on the FTCS method by using first spatially discretizing in space using *Method of Lines* (MOL) to obtain a system of ODEs to interpret the method as a time integration methods of ODEs.
For the simple heat equation $u_t = u_{xx},\ u(t=0) = u_0$, the MOL for FTCS yields a semi-discrete ODE:

$$u_t = \frac{1}{h^2}(U^n_{m-1} - 2U^n_m + U^n_{m+1}), \quad m = 1, \dots, M. \tag{66}$$

Convergence

The Dahlquist Equivalence Theorem is the primary tool for assessing whether or not a numerical scheme is convergent. Using the concepts of consistency and zero stability alone, we can draw a conclusion about the convergence.

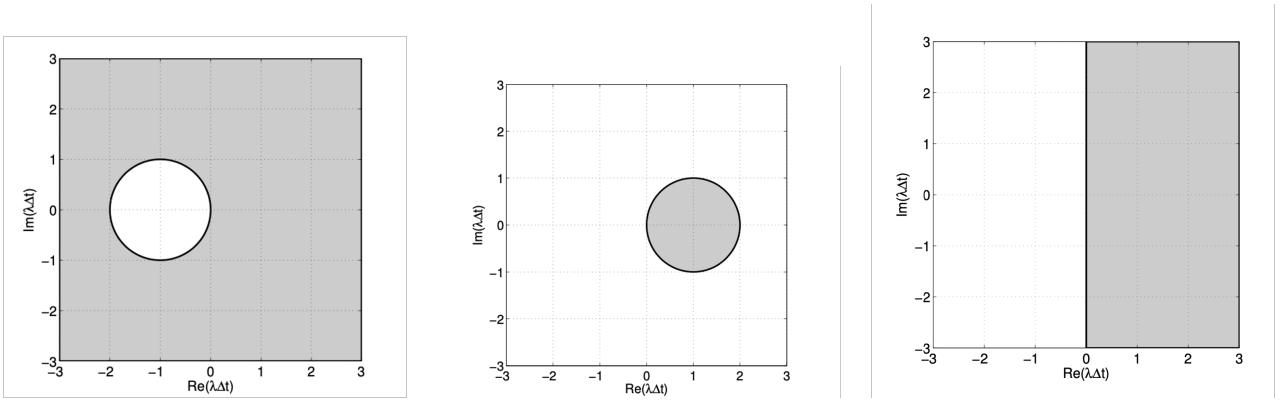


Figure 3: A comparison of the stability regions for 1) Forward Euler, 2) Backward Euler and 3) Trapezoidal where the white area corresponds to stability (the absolute stability region) and the gray area to instability [**?**]

To summarize from above, we have the following concepts:

- Consistency: In the limit $k \to 0$, the method gives a consistent discretization of the ordinary differential equation.
- Zero stability: In the limit $k \to 0$, the method has now solutions that grow unbounded as $N = T/k \to \infty$.

The Dahlquist Equivalence Theorem guarantees that a method is consistent and stable is convergent, and also that convergent method is consistent and stable.

3 AI: Large-Language Models

3.1 The Large Language Model Pipeline

The following section is a compilation of my notes over the years about LLMs and transformer architecture, much of which is based on miscellaneous papers and lessons provided by Andrej Karpathy.

Large-Language Models have been surprisingly interesting from many vantage points. Initially, I had strong biases against the idea of universal function approximators, but as I have gone deep in the topic over the past couple years, I have come around around to effectiveness of these large models.

My initial biases against deep learning were primarily fueled by my background in traditional engineering. For over a century, physicists have built foundational models from first-principles, rigorously proving the underlying mathematics and empirically validating through physical experiments. The resulting product is a framework for analyzing the world with a set of hierarchically compressed models, efficiently parameterized with falsifiable tests. Deep learning on the other hand appeared to build a model using a over parameterized universal function approximator, and using sad first-order optimization algorithms to rediscover models that the greatest minds already spent a century discovering.

Over years, the usefulness of deep learning has been undeniable. My initial thoughts on over-parameterization were overwritten with realizations of generalizability. My former disgust in brute-force gradient descent has been replaced with resulting models that handle higher degrees of complexity.

I first started playing around with DALLE 2 mid 2022, and was floored by a link sent by my friend to an introduction to my own dissertation not written by me, but by ChatGPT3. Since then, I have oscillated between skepticism in the obvious lack of intelligence and the shock from the astounding coding feats that these LLMs have provided in my academic and professional work. The pivotable moment was when our team required mundane front-end work, and a state-of-the-art model was able one shot a front-facing GUI containing 2500 lines of code.

Beyond the training, the deployment of LLMs has also caught my attention. The trillions of operations required for inference boil down to the simplest of mathematical operations, and the most critical being matrix multiplication. The scale of these matrix multiply is massive and imbalance. As model sizes range from 30b-1t in size, the deployment can range from local operations on your Apple Silicon shared memory (I am currently running 30b parameters locally on my 48GB unified share memory M4 Pro), to a fully AWS/GCP node equipped with 8 H100s with a total 640GB of VRAM. In the distributed case, this of course comes with the added complexity of distributing the inference/training on all of these GPUs and managing the communication.

Then in February 2025, the DeepSeek R1 paper launched. I remember being not very impressed with their RL approach, but their paper did spark enough interest for me to crack open their V3 Base Model Technical Report. This paper blew my mind. At the time, I had just come out of a mixed-precision computing project at work that left me in awe of the engineering feats pulled in the V3 report.

Over the last couple of years, I have done some deep dives into the topic to ensure that I am on top of the modern techniques. I typically keep my notes physical notebooks, and then eventually compile them in this PDF for future reference. The following section are my notes over the years on the topic of LLMs.

3.2 Stage 1: Dataset for Pretraining

Over the last couple of decades, there has been a large open source effort by an organization called "Common Crawl" with the purpose of scouring the Internet to build a free, open repository of the world's textual knowledge that can be used by anyone. For an lab developing their own models, this is a common starting point. Every couple of months, a new "crawl" will be released containing roughly 200-400 TiB textual content. These crawls start with a few seed pages and these seed pages follow the links and span the Internet.

However, this will not be enough, particularly since the results of the common crawl are quite rough. Organizations using Common Crawl as a starting point will need to implement their own methods of crawling, as well as filtering the text to ensure a quality dataset for training the LLM. Filtering includes removing content that may contain commonly blacklisted content (see example University of Toulouse Capital's Blacklist).

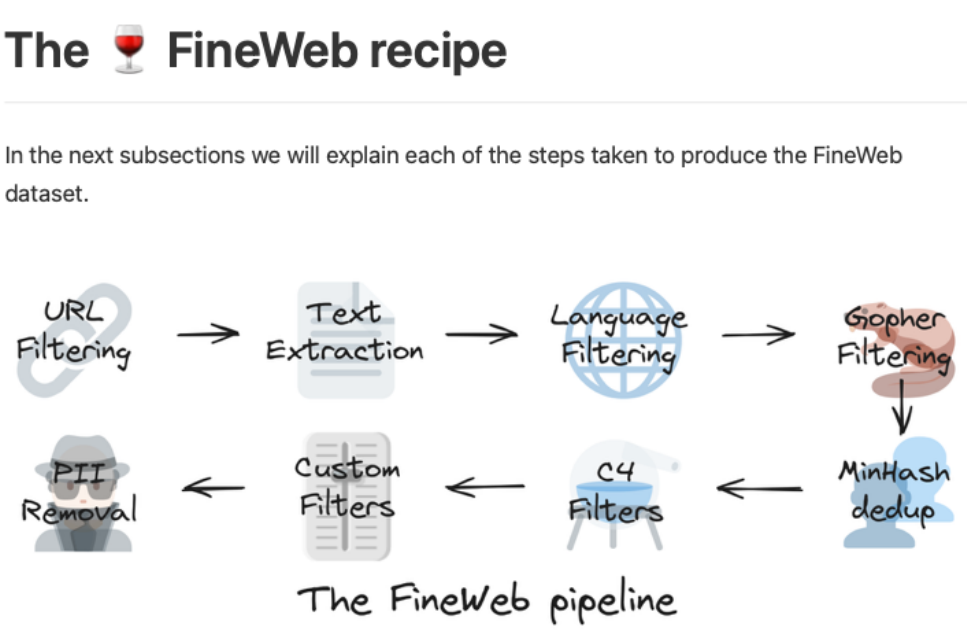


Figure 4: The Hugging Face process for filtering Common Crawl.

3.3 Stage 2: Tokenization

Once a refined representation of the Internet has been made, the next step is to represent this text in some sort of way for a neural network to process. Since the inputs to a neural network need to be numerical, we need some way to represent the text in terms of numerical values.

One way we can do this is by taking a byte, or 8 bits to sequentially represent different characters in an alphabet. With 8 bits, we get $2^8 = 256$ possible representations as numerical values of 0-255. One particular model would assign each character from the Latin-script alphabet a value. Including other common punctuation, such as space, period, etc, one could follow the patterns '0=space', '1=a', '2=b', etc.

In reality, there are several official formats that have evolved over the years. ASCII is a 7-bit encoding that enables the representation of 128 characters, and ISO 8859-1 uses 8-bit encoding. However, to extend this process across multiple languages and the infinitely expanding base of emojis, the UTF-8 encoding scheme uses 1-4 byts to represent Unicode points. Each byte has a prefix to indicate its role:

- 1-byte characters (ASCII): '0xxxxxxx'
- 2-byte: '110xxxxx' '10xxxxxx'
- 3-byte: '1110xxxx' '10xxxxxx' '10xxxxxx'
- 4-byte: '11110xxx' '10xxxxxx' '10xxxxxx' '10xxxxxx'
- continuation byte

For example, an ASCII character is represented as a single byte: 01100001, or 97. A non-ASCII character 'é' is represented by the two bytes: '11000011' '10101001', or [195, 169]

The prefix bits in each byte tell you whether this is a single-byte or multi-byte character, if multi-byte is this a first or continuation byte. This format enables UTF-8 to encode over a million characters.

This means that the raw text can be viewed in several ways. It can be viewed at the bit level, the byte level, and the UTF-8 text interpretation level. Tokenization occurs at the byte level, meaning that values of 0-255 are the initial building blocks:

```
124 86 105, 101, 119 105 110 103 3283 105 110 103 108 101 32 80 111 115 116 32
```

Byte-Pair Encoding (BPE) Algorithm

- GPT3: 50,257 tokens
- GPT4: 100,277 tokens

However, once we will see in my notes on the transformer architecture, the length of the sequence being passed into is a precious commodity, as well as the numerical range (e.g., 0-255 or 0-1e5). To increase the numerical range, new tokens can be "minted" out of combinations of common tokens that appear. For example, the byte pair '116 32' may occur often enough to justify the new token value of 256. The next most common byte pair could be assigned 257, and so on. It is unclear (to me) if this trade-off between numerical representation and sequence length is convex, but it appears that number of tokens (or numerical range) has stabilized out at 100,277 for GPT4. If someone knows if it is still increasing then please let me know.

3.4 Stage 3: LLM Training

Given a sequence of the tokens $[t_1, t_2, \ldots, t_n]$, the transformer architecture simply predicts the next token t_{n+1}. For a given piece of text "All that glitter is not gold.", we can maximize training efficiency by breaking up this single sentence into multiple examples for the LLM to train on.

$$\begin{aligned}
\text{A} &\rightarrow \text{'l'} \\
\text{Al} &\rightarrow \text{'l'} \\
\text{All} &\rightarrow \text{' '} \\
\text{All} &\rightarrow \text{'t'} \\
\text{All t} &\rightarrow \text{'h'} \\
\text{All th} &\rightarrow \text{'a'} \\
\text{All tha} &\rightarrow \text{'t'} \\
\text{All that} &\rightarrow \text{' '} \\
\text{All that} &\rightarrow \text{'g'} \\
&\vdots
\end{aligned}$$

This means, for a piece of text of length n tokens, we really have 'n-1' training examples.

3.4.1 Cross-Entropy Loss Function, a history

The goal of our generative model is learn an approximation of the distribution of all possible sequences of tokens P_{data}. Our model, P_θ is a large transformer parameterized by θ, in which we would like to "learn" the weights via training on sequences of tokens from our corpus.

Shannon Entropy - Optimal Compression

Claude Shannon's foundational work on information theory was born out of a 1948 Bell Labs paper "A Mathematical Theory of Communication". During that time, Bell Labs was attempting to scale telephone communication across the states. One of the critical problems at the time was estimating information content within a noisy message. Shannon was working on several key ideas: how much information does a message contain, how efficiently can we encode it, what is the fundamental limit of compression.

The idea of physical entropy from thermodynamics had reached maturity, to which Shannon took inspiration. Shannon argued that entropy existed in an information sense and defined $H(P)$ to be a discrete probability distribution P as:

$$H(X) = -\sum_{i=0}^{n} p_i(x) log_2 p_i(x)$$

were X is a discrete random variable that takes values $\{x_1, x_2, \ldots, x_n\}$ with probabilities $\{p_1, p_2, \ldots, p_n\}$.

This result was derived from three basic axioms of "information content":

- **Continuity**: H should be continuous over the space of all probabilities. Small in changes in probabilities leads to small changes in information
- **Monotonicity**: Rare events are more informative. If p_i is small, learning $X = x_i$, is surprising and informative.
- **Additivity**: For independent events $H(AB) = H(A) + H(B)$. In other words, information from independent sources should be additive.

The core idea of Shannon's interpretation of quantifying the entropy of information is estimating the optimal number of bits that encode a discrete probability distribution, or the information content of outcome x_i: $I(x_i) = -log_2 p_i$. For example, encoding the outcomes for the following discrete probabilities:

- A outcome of probability $P(x) = 1$ (100% certain) requires $-log_2 P(x) = 0$ bits to encode
- A outcome of probability $P(x) = 1/2$ (50% certain) requires $-log_2 P(x) = 1$ bits to encode
- A outcome of probability $P(x) = 1/4$ (25% certain) requires $-log_2 P(x) = 2$ bits to encode
- A outcome of probability $P(x) = 1/8$ (12.5% certain) requires $-log_2 P(x) = 3$ bits to encode
- ...

For example, of a fair coin toss, $p_0(H) = p_1(T) = 50\%$. Computing the Shannon entropy requires summing across all discrete scenarios:

$$H(P) = \mathbb{E}_{x\sim p}[I(x)] = \sum_{i=0}^{n} p_i(x_i) \cdot (-log_2 p_i(x_i)) = -0.5log_2(0.5) - 0.5log_2(0.5) = 1$$

bit

Or, for a biased coin – $p_0(H) = 0.9$, $p_1(T) = 0.1$:

$$H(P) = \mathbb{E}_{x\sim P}[-log_2 P(x)] = -0.9log_2(0.9) - 0.1log_2(0.1) = 0.469$$

bits

The entropy of the biased coin is lower because it is more predictable. You can compress a sequence of biased flips more efficiently.

Cross Entropy

However, rarely do we ever know the true distribution $P(x)$. Shannon proposed Q to be our estimated distribution. For outcome x_i with true probability p_i:

- Actual frequency: p_i (how often it really occurs)
- Expected code length: $-log_2 q_i$ (based on belief Q)

The expected code length is then:

$$H(P, Q) = -\sum_i p_i log_2 q_i$$

If your belief of the distribution happens to be correct ($Q = P$), then the cross entropy equation reduces to the optimal compression. In other words, $H(P, Q) \geq H(P)$, where the uncertainty in the estimated distribution results excess required bits.

Kullback-Leibler Divergence

In 1951, Kullback & Leibler provided a measure for quantifying how much Q differs from P:

$$D_{KL}(P||Q) = H(P, Q) - H(P) = \sum_i p_i log \frac{p_i}{q_i}$$

also known as the KL divergence. This measure has the following key properties:

- $D_{KL}(P||Q) \geq 0$
- $D_{KL}(P||Q) = 0$ if and only if $Q = P$

- $D_{KL}(P||Q) \neq D_{KL}(Q||P)$

A Cost Function for LLM Training

The true distribution of tokens for natural language P is unknown, and our belief of the distribution is parameterized by Q_θ, where θ are the model weights. To update θ to better match the true distribution, we ideally need to minimize $D_{KL}(P||Q_\theta) = H(P, Q_\theta) - H(P)$. Since $H(P)$ is constant relative to θ, $\arg\min_\theta D_{KL}(P||Q_\theta) = \arg\min_\theta H(P, Q_\theta)$.

In the case of natural language, the actual distribution of next token output from a human is the true distribution:

$$P(t_i \mid t_1, \ldots, t_{i-1})$$

where a LLM output is our belief of the distribution:

$$Q_\theta(t_i \mid t_1, \ldots, t_{i-1}) = \text{softmax}(\text{model\_output})$$

.

The cross-entropy at position i is:

$$H(P, Q_\theta)_i = -\sum_{t \in V} P(t \mid t_{<i}) log Q_\theta(t \mid t_{<i})$$

where V is the vocabulary. During training, when a sequence is passed into model, it is compared to example which is an specific target token t_i^*:

$$P(t \mid t_{<i}) = \begin{cases} 1 & \text{if } t = t_i^* \\ 0 & \text{otherwise} \end{cases}$$

collapsing the summation across all terms to:

$$H(P, Q_\theta)_i = -1 \cdot log Q_\theta(t_i^* \mid t_{<i})$$

where t_i^* the probability of the correct token output from the softmax distribution.

3.5 Stage 4: Inference

3.6 Stage 5: Post Training

3.7 Transformer Architecture

The transformer sought to solve fundamental issues with previous natural language approaches, specifically on how words or tokens relate to each other. "Attention is All You Need" solved this by proposing mechanism that builds an attention (correlation) matrix between each of the tokens in a sequence, providing a learned similarity score based on projected embeddings. The mechanism is baked into a larger neural network architecture that has proved to be scalable and generalizable. My notes begin with the entry point of this architecture.

3.7.1 Word and Positional Embeddings

To capture linguistic meaning behind a token, each scalar mapping between 0-100,276 for GPT4 a higher dimensional vector space better suited for deep learning. The vector is referred to as the token's embedding (or colloquially word embedding), which can be learned during the training process.

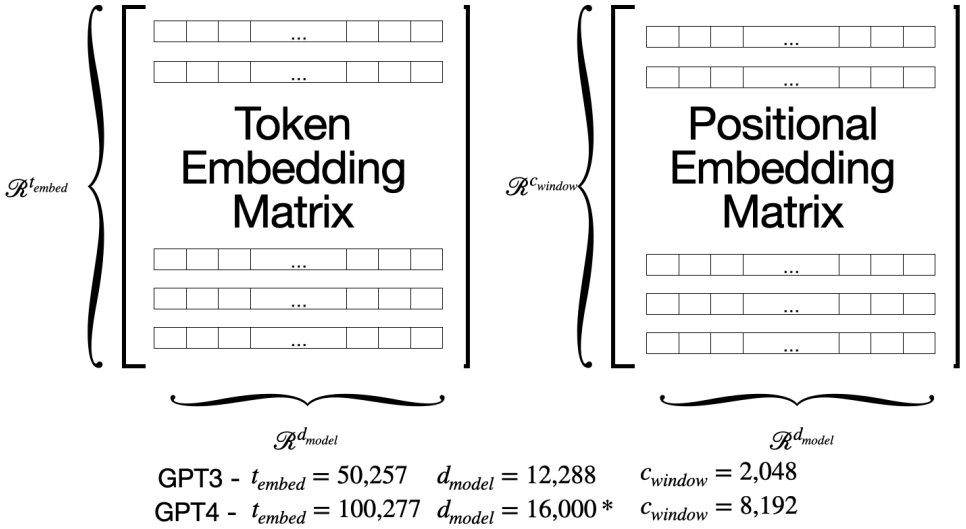


Figure 5: The embedding matrix dimensions.

No matrix operations occurs at this point, rather the token's integer value is used to index into the embedding matrix to pluck out a row.

The algorithmic information content of a scalar integer representational is limited to a token's literal representation, which compresses any semantic or contextual meaning out. By mapping the scalar representation of each token into a higher dimensional space, semantic and contextual meaning can be captured and stored as relative weights. Provided many examples, an empirically-derived model of how tokens correlate with one another can be statistically determined. In other words, we can represent a token or word as a higher dimension object, and "learn" the parameters from corpus. By systematically experimenting with training algorithms, our hyperparameter dimension size can also be tuned to ensure that the algorithmic content of the vector is sufficiently compressed while still being sufficiently expressive. For LLMs, these dimension has been set as $\boldsymbol{t} \in \mathcal{R}^{d}_{model}$, where $d_{model} = 12,288$ for GPT3 and $d_{model} = 16,000$ for GPT4. A great blog post by Hugging Face describes the techniques in learning these matrices under different applications: Primer on embeddings. These techniques extend beyond natural language processing and also exist for applications in image, video, sound and the physical world.

Although the efficiency of the algorithmic content of a vector remains to be proven, the semantic meanings behind these embeddings provide a vector space that relate tokens of categorically similar meanings closer to one another. The vectors learned in embedding matrix are considered static, but still provide quite a bit of information about the token. 3 Blue 1 Brown has an excellent visualization of vector embeddings describing how vectors representing words such as queen and king have similar vectorial differences as the differences between other gender specific words.

These token representations need not be static. In fact the transformer architecture's purpose is to process information about the sequence and update each tokens embedding based off of this information. As each embedding flows through the network, the vector is updated to a more contextually rich representation of the token in the provided sequence.

3.7.2 The Positional Update

The first embedding transformation incorporates a token's position in the sequence. The way this is done has varied since the original transformer paper, but as of today it is common to directly "learn" this matrix. The dimensions of this matrix are slightly different than the token embedding matrix. The number of rows corresponds to the maximum sequence length (context window), while the number of columns matches the token embedding dimension (`d_model`) so that addition is valid. The original GPT3 model had a context window of 2,048 tokens, which is roughly 1,500 words. The original GPT4 model bumped that size up to 8,192 tokens, with subsequent models substantially bumping those numbers. Gemini 2.5 Pro (as of 2025) has a gargantuan context window size of 1 million tokens. It seems a bit silly to go into details on the context windows of any models today since these are being updated so fast that it is difficult to keep up.

For each token, its position in the sequence determines which row of the positional embedding matrix is selected. This vector is then added to the token embedding, similar to how token embeddings are selected. Visually, this is represented in Figure 6.

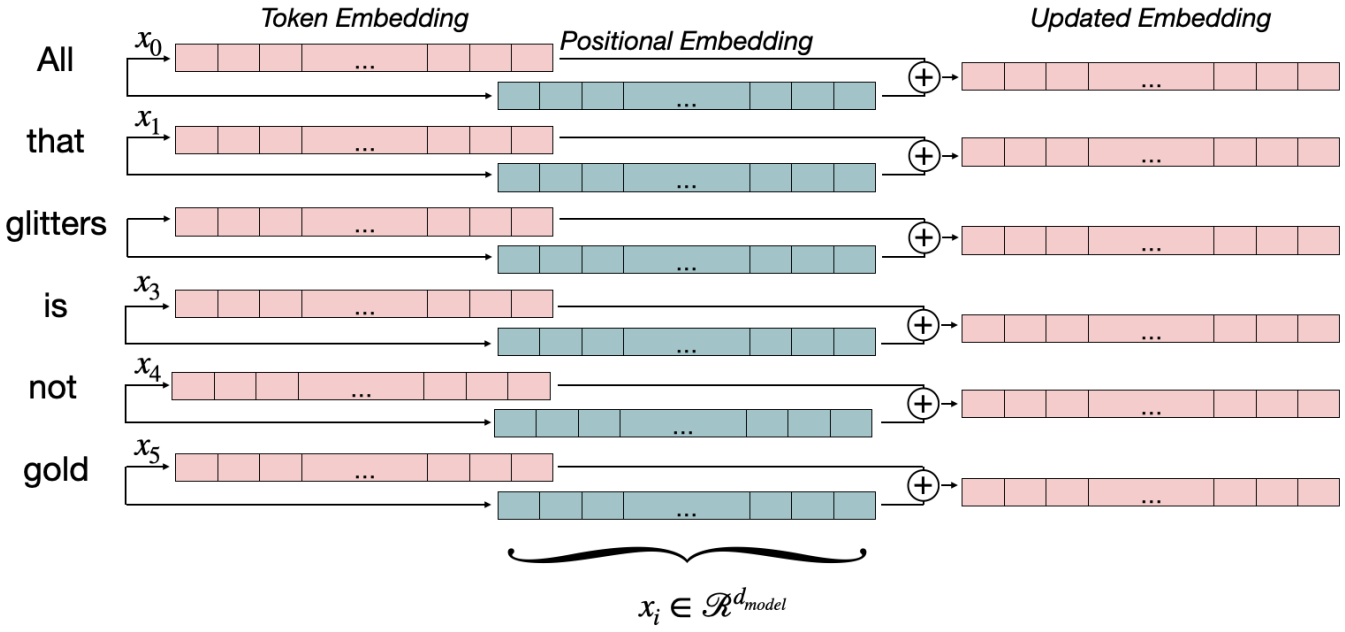


Figure 6: A token's embedding vector is "plucked" from the token embedding matrix, and its corresponding positional vector is also "plucked from the positional embedding matrix. These two vectors are added together and are the first update in the transformer architecture that adds context to token embeddings vectorial representation.

The output embedding transformation is a matrix of size $\mathcal{R}^{s_{length} \times d_{model}}$. One way to think about each row vector in this matrix that each token embedding has been vectorially updated with new context information about the position. As with token embedding matrix, the weights of the positional embedding matrix are learned and are constant during inference.

3.7.3 Self-Attention Mechanism

The embedding matrix $\boldsymbol{X} \in \mathbb{R}^{s_{\text{length}} \times d_{\text{model}}}$ is linearly projected in three different ways: Keys $\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K^\top$, Queries $\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q^\top$, and Values $\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V^\top$. I define these projections with transposes so that the output matrices preserve the same row-wise structure as X — pedagogically convenient when following along with Figure 7. This maps embeddings from $\mathbb{R}^{s_{\text{length}} \times d_{\text{model}}}$ to a lower dimensional space $\mathbb{R}^{s_{\text{length}} \times d_{\text{head}}}$ (more on this in the section on multi-head attention).

The matrices $\boldsymbol{K}$, $\boldsymbol{Q}$, and $\boldsymbol{V}$ are often semantically described with some notion of "mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors." While the original transformer paper offered this as one possible interpretation, subsequent academicians, blogsters, and yada yada-ers have continued to theory-wash the transformer architecture with database metaphors.

Mechanistically, the attention score matrix $\boldsymbol{Q}\boldsymbol{K}^\top$ is simply the collection of pairwise inner products between two learned projections of the same input. The apparent asymmetry is not a profound statement about semantics – it arises only because the model is granted two separate projection matrices, giving it extra degrees of freedom to sculpt similarity patterns in representation space.

$$\sigma(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_{head}}})$$

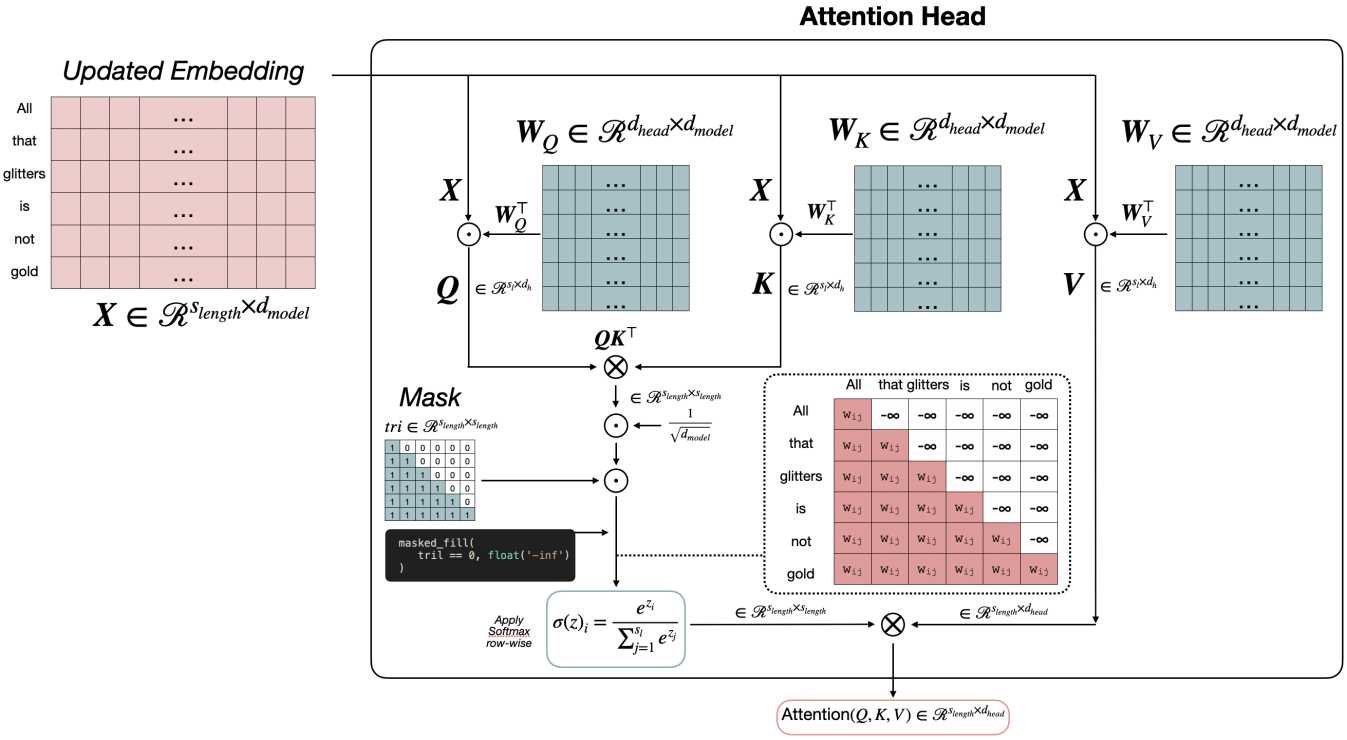


Figure 7: A single attention head.

3.7.4 Multi-Head Attention

A single head of attention naturally emphasizes a small subset of highly correlated pairs due to softmax's exponential weighting by sharpening the distribution, so a single head can collapse to a dominant pattern. Multi-head attention solves this by running h independent attention heads in parallel, each with its own learned projection matrices $\boldsymbol{W}_Q^{(i)}, \boldsymbol{W}_K^{(i)}, \boldsymbol{W}_V^{(i)}$ for $i = 1, \ldots, h$. Each head of attention is then capable of emphasizing possible candidate correlated pairs within subset of the embedding space.

Computationally, each head produces an output $\boldsymbol{A}_{d_i} \in \mathbb{R}^{s_{length} \times d_{head}}$, where $d_{head} = d_{model}/h$. These head outputs are concatenated along the feature dimension to form:

$$\boldsymbol{A} = \text{Concat}(\boldsymbol{A}_1, \ldots, \boldsymbol{A}_h) \in \mathbb{R}^{s_{length} \times d_{model}}.$$

A final learned linear projection $\boldsymbol{W}_O \in \mathbb{R}^{d_{model} \times d_{model}}$ maps the output $\boldsymbol{A}$ back to a unified embedding space:

$$\text{MHA}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \boldsymbol{A}\boldsymbol{W}_O.$$

Each head attends over the same input but may learn different attention distributions: some specialize in short-range dependencies, others in long-range interactions. The concatenation and output projection then fuse these differences. Figure 8 illustrates this process: multiple attention heads operate independently on the same input embeddings $\boldsymbol{X}$, producing head-specific outputs $\boldsymbol{A}_{d_i}$ that are then stitched together to form the final updated embedding matrix $\boldsymbol{A}$.

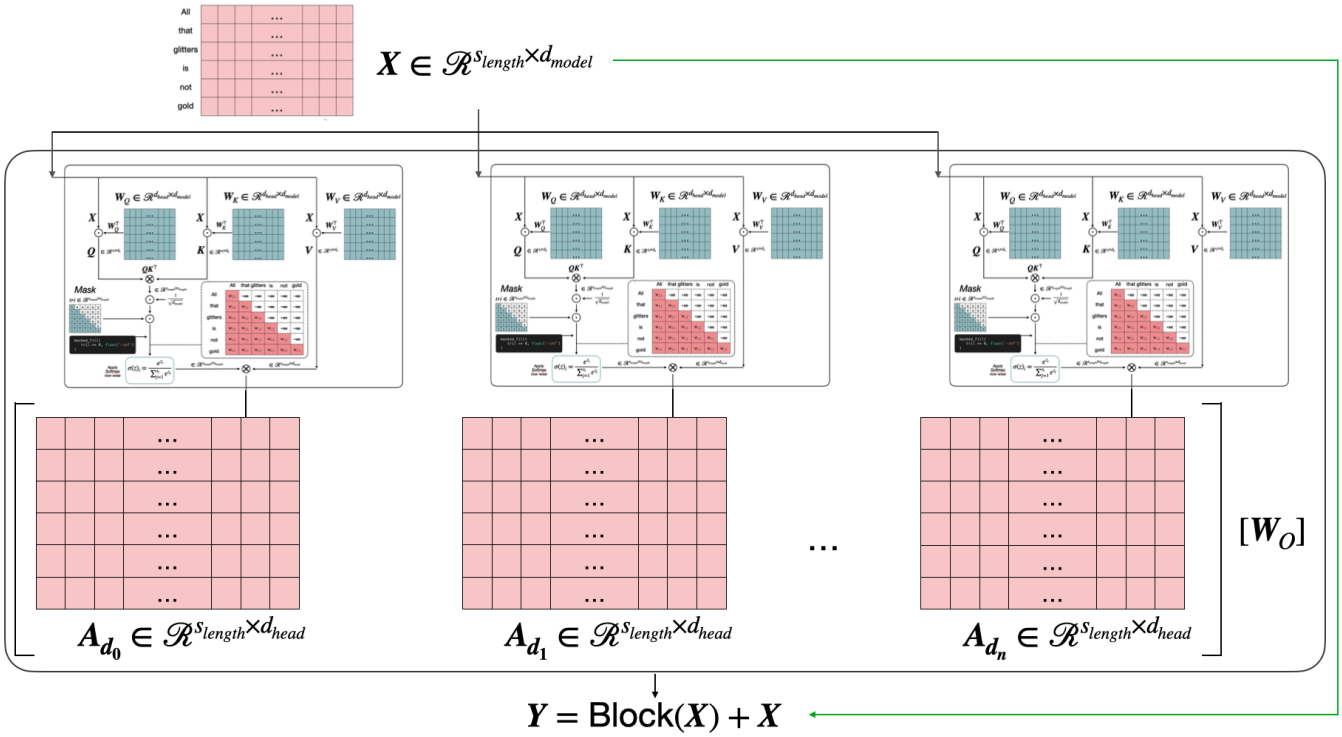


Figure 8: Multiple heads of attention followed by residual connection and layer normalization.

At this point, the transformer has enriched token embeddings with contextual information by passing them through layers of self-attention. However, attention alone is not enough: it mixes information across tokens, but it does not transform that information in richer, nonlinear ways. To build depth and expressive power, the transformer architecture alternates attention layers with position-wise feedforward networks, all stabilized with residual connections and normalization. This is where the rest of the architecture comes in.

3.7.5 Position-wise Feedforward Network

Self-attention enriches token embeddings with contextual information, but by itself it is not a universal function approximator. The inner products and softmax are essentially linear similarity operations with a nonlinearity for weighting. To enrich the representational capacity of each token, the transformer alternates attention layers with a position-wise feedforward network (FFN).

The FFN is applied independently to each token embedding, without mixing across the sequence dimension. For a given input token embedding $\boldsymbol{x} \in \mathbb{R}^{d_{model}}$, the block is defined as:

$$\text{FFN}(\boldsymbol{x}) = \sigma(\boldsymbol{x}\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2$$

where $\boldsymbol{W}_1 \in \mathbb{R}^{d_{model} \times d_{ff}}$, $\boldsymbol{W}_2 \in \mathbb{R}^{d_{ff} \times d_{model}}$, and d_{ff} is typically larger than d_{model} (for GPT-3, $d_{ff} \approx 4 \times d_{model}$). The activation function σ was chosen as ReLU in the original transformer, but variants such as GELU and SwiGLU have since become standard in large language models. This feedforward block provides depth and nonlinear expressivity at the level of individual tokens.

Because the FFN operates identically and independently on each position, it is sometimes called a **position-wise feedforward network**. This design keeps computation efficient: the same learned transformation is applied independently and in parallel across all tokens.

3.7.6 Residual Connections and Layer Normalization

Each attention and feedforward block in the transformer is wrapped in a residual connection followed by a normalization step. These design choices enable numerical stability at scale. This secondary path (labeled in green in Figure 8 ensures that the gradients can flow directly through the network, mitigating the risk of vanishing/exploding gradi-

ents. In practice, residuals make it possible to stack dozens or hundreds of transformer layers without optimization failure.

The residual connection adds the input of a block back to its output:

$$\boldsymbol{Y} = \boldsymbol{X} + \text{Block}(\boldsymbol{X})$$

After the residual addition, the combined vector is normalized using **layer normalization**:

$$\text{LayerNorm}(\boldsymbol{Y}) = \frac{\boldsymbol{Y} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \cdot \boldsymbol{\gamma} + \boldsymbol{\beta}$$

where $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are the mean and standard deviation of the features in $\boldsymbol{Y}$, and $\boldsymbol{\gamma}, \boldsymbol{\beta}$ are learnable scale and shift parameters. Unlike batch normalization, which normalizes across examples in a batch, layer normalization operates along the feature dimension of a single token embedding. This makes it naturally compatible with sequence models, where batch statistics may vary widely.

The original transformer paper applied normalization *after* the residual addition (known as **post-norm**). Many modern architectures flip the order, applying normalization *before* the block (**pre-norm**), which empirically improves training stability for very deep models. The distinction is subtle but has large implications for scaling.

Figure **??** illustrates the full transformer block: a self-attention module and a feedforward network, each wrapped in residual connections and normalization. This design makes the block composable: attention mixes information across tokens, the feedforward expands nonlinear representation within tokens, and the residual-normalization wrapper numerically stabilizes the model for training at scale.

4 Optimization

4.1 Introduction

Optimization, a concept widely applied across various disciplines, holds profound significance in mathematics and physics. The observed tendency of physical systems towards states of stable equilibrium serves as a fundamental principle underlying numerous phenomena. In physics and chemistry, dynamical systems evolve along paths that minimize their energy state, ultimately reaching an equilibrium point. This paradigm extends to diverse fields such as finance, engineering, operations research, and parameter estimation, where the objective is to maximize or minimize a specific quantity by manipulating decision variables (also referred to as optimization variables or manipulated variables).

The optimization process can manifest in different forms. In some instances, systems converge to a local minimum energy state following a descent path, analogous to a ball rolling down a hill to settle in a valley. In other cases, optimization emerges from seemingly random processes, as exemplified by the evolutionary principle of survival of the fittest in Darwinian theory.

Mathematically, we can formulate these problem statements as a mapping between inputs $\boldsymbol{x} \in \mathcal{X}$ and outputs $\boldsymbol{y} \in \mathcal{Y}$, represented by a function $f : \mathbb{R}^n \to \mathbb{R}$. The objective of mathematical optimization is to identify an input $\boldsymbol{x} \in \mathbb{R}^n$ that minimizes or maximizes the output $y \in \mathbb{R}$. The standard form of such an optimization problem can be expressed as follows:

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & f_0(\boldsymbol{x}) \\ \text{subject to} \quad & f_i(\boldsymbol{x}) \leq b_i, \quad i = 1, ..., m. \end{aligned}$$

which reads as "minimize $f_0(\boldsymbol{x})$ subject to $f_i(\boldsymbol{x}) \leq b_i$ for all $i = 1, ..., m$," where $f_0(\boldsymbol{x})$ is the objective function $f_0 : \mathbb{R}^n \to \mathbb{R}$ (also referred to as the cost or evaluation function), $\boldsymbol{x} \in \mathbb{R}^n$ is the vector of decision variables, $f_i : \mathbb{R}^n \to \mathbb{R},\ i = 1, ..., m$ are the inequality constraints, and $b_1,\ ..., b_m$ are the constraint bounds. The optimal solution is denoted as $\boldsymbol{x}^*$.

It is important to note that while the concept of optimization is broadly applicable, not all optimization problems are solvable. The tractability of an optimization problem primarily depends on the structure of the objective function $f_0(\boldsymbol{x})$. In some cases, $f_0(\boldsymbol{x})$ is an explicit function amenable to analytical operations. For instance, the univariate objective function $f_0(\boldsymbol{x}) = (x+2)^2 + 1$ can be minimized using elementary calculus techniques, yielding an analytical solution of $x^* = -2$. However, analytical solutions are often unattainable for most practical problems.

For more complex optimization problems, numerical methods provide a powerful toolset. These problems can generally be categorized into two main classes: convex optimization and stochastic (blackbox) optimization. The following sections will delve into various numerical techniques for solving tractable optimization problems within these categories, exploring their principles, applications, and limitations.

4.2 Gradient-based methods

Newton's method:

Let us first consider an convex objective $f_0(x)$ and method in which we can approach solving the minimization. Convexity enables us to makes certain assumptions about optimality conditions. One optimality condition being

$$\nabla_{\boldsymbol{x}} f_0(\boldsymbol{x}) = 0, \tag{67}$$

or more simply, we would like to find the root of the gradient of $f_0(\boldsymbol{x})$.

Newton's method is a simple numerical approach used for find roots of an equation if an initial candidate solution is close the exact solution. Here I derive Newton's method using a Taylor Series expansion.

Suppose $f_0(\boldsymbol{x})$ is a continuous function on a closed interval $[a, b]$ and has $n+1$ continuous derivatives on the open interval (a, b). If $x, c \in (a, b)$, then the expansion of $f_0(x)$ about c is:

$$f(c) + f'(c)(x-c) + \frac{1}{2!}f''(c)(x-c)^2 + \frac{1}{3!}f'''(c)(x-c)^3 + ... + \frac{1}{n!}f^n(c)(x-c)^n \tag{68}$$

or more concisely:

$$f(x) \approx \sum_{k=0}^{\infty} \frac{1}{k!} f^{(k)}(c)(x-c)^k \tag{69}$$

By approximating $f_0(\boldsymbol{x})$ using this expansion, we can find the optimal x^* by expanding $\nabla_x f_0(x)$ around a initial guess value x^j, which at optimum should be equal to zero:

$$\nabla_x f_0(x^{j+1}) = \nabla_x f_0(x^j) + \nabla_x(\nabla_x f_0(x^j))(x^{j+1} - x^j) + \mathcal{O}(x^{j+1} - x^j)^2 = 0|_{x^{j+1}=x^*}. \tag{70}$$

The resulting error $\mathcal{O}(x^{j+1} - x^j)^2$ indicates that this optimization method is a second-order gradient-based approach. Rearranging the above equation yields:

$$-\nabla_x f_0(x^j) = \nabla_x(\nabla_x f_0(x^j))(x^{j+1} - x^i) \tag{71}$$

where we seek to solve for x^{j+1}, where $\nabla_x(\nabla_x f_0(x^j))$ is the Hessian. If x is a vector, than a matrix inversion is required to solve for x^{j+1}.

$$\boldsymbol{x}^{j+1} = \boldsymbol{x}^j - [\nabla_x(\nabla_x f_0(\boldsymbol{x}^j)]^{-1}\nabla_x f_0(\boldsymbol{x}^j). \tag{72}$$

Explicitly, the Hessian is defined as:

$$H = \nabla_x(\nabla_x f_0(\boldsymbol{x}_i)) = \begin{bmatrix} \frac{\partial f_0^2}{\partial x_1^2} & \frac{\partial f_0^2}{\partial x_1 \partial x_2} & \cdots & \frac{\partial f_0^2}{\partial x_1 \partial x_n} \\ \frac{\partial f_0^2}{\partial x_2 \partial x_1} & \frac{\partial f_0^2}{\partial x_2^2} & \cdots & \frac{\partial f_0^2}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_0^2}{\partial x_n \partial x_1} & \cdots & \cdots & \frac{\partial f_0^2}{\partial x_n^2} \end{bmatrix} \tag{73}$$

This means that use of Newton's methods for gradient descent requires an inversion of a $n \times n$ matrix (a computational cost $\mathcal{O}(n^3)$) to compute the next candidate solution. The update equation can be explicitly referred to as:

$$\begin{bmatrix} x_1^{j+1} \\ \vdots \\ x_n^{j+1} \end{bmatrix} = \begin{bmatrix} x_1^j \\ \vdots \\ x_n^j \end{bmatrix} - \begin{bmatrix} \frac{\partial f_0^2}{\partial x_1^2} & \cdots & \frac{\partial f_0^2}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_0^2}{\partial x_n \partial x_1} & \cdots & \frac{\partial f_0^2}{\partial x_n^2} \end{bmatrix} \begin{bmatrix} \frac{\partial f_0}{\partial x_1} \\ \vdots \\ \frac{\partial f_0}{\partial x_n} \end{bmatrix}. \tag{74}$$

Sometime, we do not have an explicit form of $f_0(\boldsymbol{x})$ and therefore do not have access to the gradients directly. We can approximate these gradients numerically (see Section on Numerical Methods), however Newton's method quickly deteriorates as a viable option as $n \to \infty$ since matrix inversion has computational complexity on the order of $\mathcal{O}(n^3)$.

First-order methods

Gradient descent optimizes a function by using first-order gradient information, while Newton's method optimizes a function utilizing second-order gradient information. The most basic implementation of gradient descent computes the gradient of the objective function and slightly perturbs the current position. Although first-order methods are considerably faster, these algorithms can be unstable and perform poorly on non-convex functions. Consider the example in which parameters of a model are being tuned to fit a data set:

Let θ be the parameters for a function approximator $f_0(x)$. Gradient descent can computed iteratively as

$$\theta^{i+1} = \theta^i - \gamma \nabla_\theta f_0(x), \quad x \in \mathcal{D} \tag{75}$$

where γ is a hyper parameter chosen to determine the incremental step size performed by optimizer.

Extensions of First-Order Methods First-order methods can be improved by incorporating additional information about past gradients or modifying the learning rate dynamically. Here I discuss a few popular extensions: Gradient Descent with Momentum, Nesterov Accelerated Gradient, and Adaptive Gradient Algorithms (such as AdaGrad, RMSprop, and Adam). See **??** for basic python implementations and examples.

4.2.1 Gradient descent with momentum

Momentum is a method that helps accelerate gradients vectors in the right directions, thus leading to faster converging. The momentum method introduces a velocity vector that accumulates past gradients to smooth out the update process.

The update rule for gradient descent with momentum is:

$$\begin{aligned} \boldsymbol{v}^i &= \beta \boldsymbol{v}^{i-1} + (1-\beta)\nabla_\theta f_0(\theta^i) \\ \theta^{i+1} &= \theta^i - \gamma \boldsymbol{v}^i \end{aligned} \tag{76}$$

where β is the momentum term (typically set between 0.8 and 0.9) and $\boldsymbol{v}$ is the velocity vector.

4.2.2 Nesterov Accelerated Gradient

Nesterov Accelerated Gradient (NAG) is a modification of the momentum method that improves the convergence rate by making a correction before the gradient is evaluated.

The update rule for NAG is:

$$\begin{aligned} \boldsymbol{v}^i &= \beta \boldsymbol{v}^{i-1} + (1-\beta)\nabla_\theta f_0(\theta^i - \gamma\beta \boldsymbol{v}^{i-1}) \\ \theta^{i+1} &= \theta^i - \gamma \boldsymbol{v}^i \end{aligned} \tag{77}$$

4.2.3 Adaptive Gradient Algorithms

Adaptive gradient algorithms adjust the learning rate based on the past gradients' magnitudes, allowing for more fine-tuned updates. Here we introduce a few popular adaptive methods:

AdaGrad: AdaGrad adapts the learning rate for each parameter based on the past squared gradients, allowing for larger updates for infrequent parameters and smaller updates for frequent ones.

The update rule for AdaGrad is:

$$\begin{aligned} \boldsymbol{G}^i &= \boldsymbol{G}^{i-1} + \nabla_\theta f_0(\theta^i) \odot \nabla_\theta f_0(\theta^i) \\ \theta^{i+1} &= \theta^i - \frac{\gamma}{\sqrt{\boldsymbol{G}^i + \varepsilon}} \odot \nabla_\theta f_0(\theta^i) \end{aligned} \tag{78}$$

where $\boldsymbol{G}$ is the accumulation of past squared gradients and ε is a small constant to avoid division by zero.

RMSprop: RMSprop addresses AdaGrad's issue of rapidly diminishing learning rates by introducing a decay factor.

The update rule for RMSprop is:

$$\begin{aligned} \boldsymbol{G}^i &= \beta \boldsymbol{G}^{i-1} + (1-\beta)\nabla_\theta f_0(\theta^i) \odot \nabla_\theta f_0(\theta^i) \\ \theta^{i+1} &= \theta^i - \frac{\gamma}{\sqrt{\boldsymbol{G}^i + \varepsilon}} \odot \nabla_\theta f_0(\theta^i) \end{aligned} \tag{79}$$

Adam: Adam (Adaptive Moment Estimation) combines the ideas of momentum and RMSprop by maintaining both first and second moment estimates of the gradients. The original paper "ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION" has well over 100,000 citations!

The update rule for Adam is:

$$\begin{aligned} \boldsymbol{m}^i &= \beta_1 \boldsymbol{m}^{i-1} + (1-\beta_1)\nabla_\theta f_0(\theta^i) \\ \boldsymbol{v}^i &= \beta_2 \boldsymbol{v}^{i-1} + (1-\beta_2)\nabla_\theta f_0(\theta^i) \odot \nabla_\theta f_0(\theta^i) \\ \hat{\boldsymbol{m}}^i &= \frac{\boldsymbol{m}^i}{1-\beta_1^i} \\ \hat{\boldsymbol{v}}^i &= \frac{\boldsymbol{v}^i}{1-\beta_2^i} \\ \theta^{i+1} &= \theta^i - \frac{\gamma}{\sqrt{\hat{\boldsymbol{v}}^i + \varepsilon}} \hat{\boldsymbol{m}}^i \end{aligned} \tag{80}$$

where β_1 and β_2 are hyperparameters typically set to 0.9 and 0.999, respectively, and ε is a small constant.

An example implementation in Python is shown below while attempting to optimize the non-convex Beale Function:

```
 # manual gradients and hessians for each funciton
def grad_beale(curr_loc, ord=1, method=1):
    """ computes gradient and hessian
        inputs:
            - {x, y} coords: ndarray
            - ord: 1 or 2 for gradient or hessian
            - method: 1 or 2 for analtyical or numerical
        outputs:
            - gradient
            - hessian """
    x = curr_loc[0]
    y = curr_loc[1]

    # compute gradient
    dfdx = 2 * (1.5 - x + x * y) * (-1 + y) + \
           2 * (2.25 - x + x * y ** 2) * (-1 + y ** 2) + \
           2 * (2.6250 - x + x * y ** 3) * (-1 + y ** 3)
    dfdy = 2 * (1.5 - x + x * y) * x + \
           2 * (2.25 - x + x * y ** 2) * (2 * x * y) + \
           2 * (2.6250 - x + x * y **3) * (3 * x * y ** 2)

    gradient = np.array([dfdx, dfdy])

    if ord == 1:
        return gradient

    elif ord == 2:
        # compute hessian
        ddfddx = 2 * (y - 1) * (y - 1) + \
                 2 * (y * y - 1) * (y * y - 1) + \
                 2 * (y * y * y - 1) * (y * y * y - 1)
        ddfdydx = 4 * x * (y ** 2 - 1) + \
                  4 * y * (x * y * y - x + 2.25) + \
                  6 * x * (y * y * y - 1) * y * y + \
                  y * y ** 2 * (x * y * y * y - x + 2.625) + \
                  2 * x * (y - 1) + \
                  2 * (x * y - x + 1.5)
        ddfddy = 18 * x * x * y * y * y * y + \
                 8 * x * x * y * y + \
                 4 * x * (x * y * y - x + 2.25)

        hessian = np.array([[ddfddx, ddfdydx], [ddfdydx, ddfddy]])
        return gradient, hessian
```

```
def grad_descent(grad_fn, init_loc=[0, 0], n_steps=10000, f=None):
    gamma = 1e-6

    # initialize trajectories
    traj = np.zeros((n_steps, len(init_loc)))
    traj[0, :] = np.array(init_loc)

    for i in range(1, n_steps):
        if f is not None:
            traj[i, :] = traj[i - 1, :] \
                       - gamma * grad_fn(f=f,
                                         curr_loc=traj[i - 1, :],
                                         ord=1)
        else:
            traj[i, :] = traj[i - 1, :] \
                       - gamma * grad_fn(traj[i - 1, :],
                                         ord=1)

    return traj

for i in range(len(initial_points)):
    trajectories_gd = grad_descent(grad_beale, initial_points[i])
    output, fig, ax = beale([], [])
    ax.scatter(trajectories_gd[:-1, 0], trajectories_gd[:-1, 1],
               s=25, facecolor='orange')
    ax.scatter(trajectories_gd[-1, 0], trajectories_gd[-1, 1],
               s=150, facecolor='m', marker='*', label='gd opt')
```

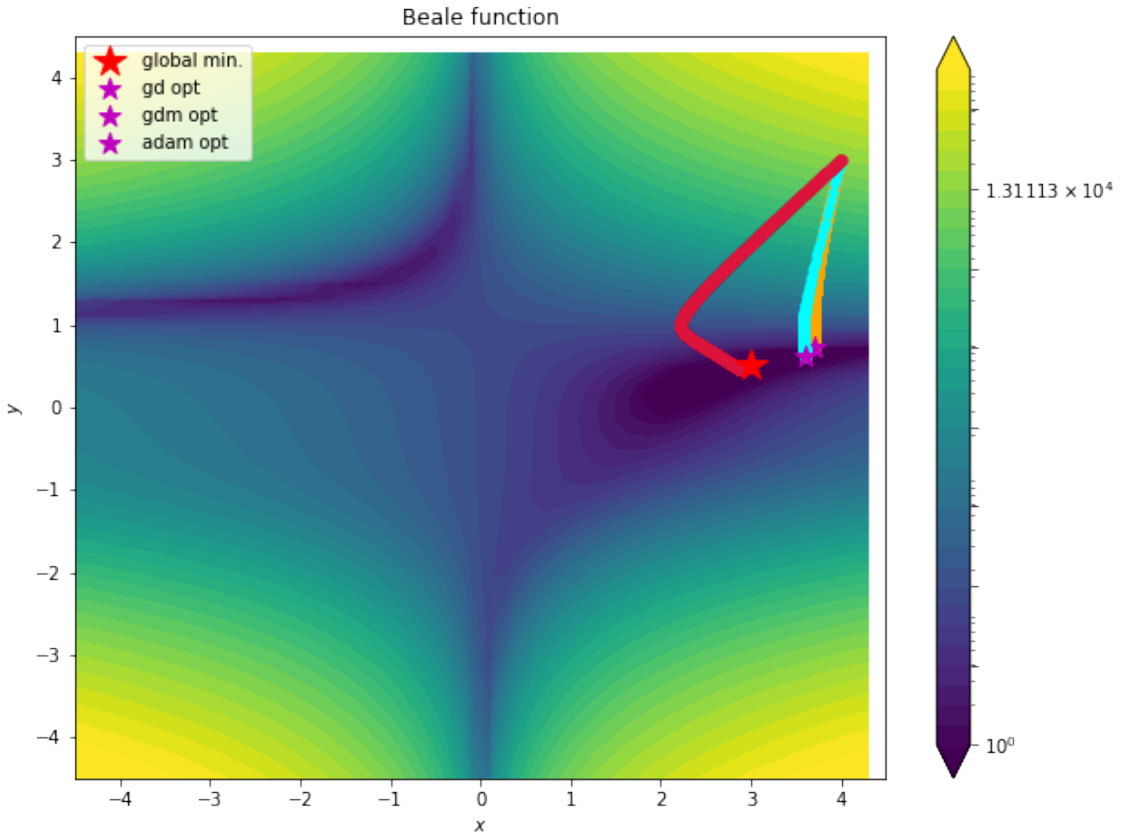


Figure 9: A benchmark comparison between gradient descent, gradient descent + momentum, and ADAptive Momentum (ADAM) gradient descent on the non-convex Beale Function

4.2.4 Blackbox optimization + non-gradient-based methods

4.3 Genetic Algorithms

Genetic Algorithms (GAs) are a subset of Machine Learning Algorithms (MLAs) that are particularly well-suited for solving nonconvex, nonsmooth, multicomponent, and multistage optimization problems. These algorithms draw inspiration from the principles of natural selection and evolution, making them robust tools for complex system optimization where traditional gradient-based methods may fail.

4.3.1 Algorithm Structure

Following the work of Zohdi [?, ?, ?, ?, ?, ?], the basic structure of a genetic algorithm can be described as follows:

Algorithm 1: Genetic Algorithm

Generate an initial population of S genetic strings **while** *termination condition not met* **do**
Compute fitness of each string
Rank genetic strings
Mate nearest pairs to produce offspring
Eliminate bottom M strings
Keep top K parents and their K offspring
Introduce M new random strings
return best solution found

4.3.2 Genetic String Representation

Each candidate solution in the population is represented as a genetic string $\boldsymbol{\lambda}_i$, defined as:

$$\boldsymbol{\lambda}_i \stackrel{\text{def}}{=} \left[\lambda_1^i,\ \lambda_2^i,\ \lambda_3^i,\ \ldots\ \lambda_N^i\right] \tag{81}$$

where each λ_j^i represents a parameter in the solution, constrained within specific ranges: $\lambda_j^{(-)} \leq \lambda_j^i \leq \lambda_j^{(+)}$.

4.3.3 Mating and Offspring Generation

The algorithm generates offspring through a mating process between pairs of parent strings. For a pair of parent strings $\boldsymbol{\lambda} i$ and $\boldsymbol{\lambda} i + 1$, two offspring are produced:

$$\boldsymbol{\lambda}_i^{\text{new}} = \boldsymbol{\phi} \odot \boldsymbol{\lambda}_i + (1 - \boldsymbol{\phi}) \odot \boldsymbol{\lambda}_{i+1} \tag{82}$$

$$\boldsymbol{\lambda}_{i+1}^{\text{new}} = \boldsymbol{\psi} \odot \boldsymbol{\lambda}_i + (1 - \boldsymbol{\psi}) \odot \boldsymbol{\lambda}_{i+1} \tag{83}$$

where $\boldsymbol{\phi}$ and $\boldsymbol{\psi}$ are vectors of random numbers between 0 and 1, and $\odot$ denotes the Hadamard (element-wise) product.

4.3.4 Key Observations and Best Practices

Hybrid Approach: It's often effective to use a GA to isolate multiple local minima, then apply gradient-based methods in these locally convex regions or reset the GA to focus its search around the best-performing parameter sets.

Mutation: While it's possible to induce mutations by selecting mating parameters outside the [0,1] range, this is somewhat redundant with the introduction of new random members in each generation.

Parent Retention: Keeping top-performing parents in the next generation ensures monotonic reduction of the cost function and reduces computational cost, as retained parents don't need re-evaluation.

Population Size: Sufficiently large population sizes help mitigate potential issues with inbreeding when retaining parents.

4.4 Bayesian Optimization (BO)

The following are my personal notes on the topic of BO.

Bayesian optimization is an efficient strategy for the optimization of expensive black-box functions. It is particularly useful when function evaluations are costly, time-consuming, or have limited availability. The core principle of BO is to construct a probabilistic model (surrogate model) of and leverage this model to intelligently select the most promising points for subsequent evaluation. This section gives a broad overview of core concepts in BO, specifically the various approaches to surrogate modeling and manage the Exploration/Exploitation Dilemma.

4.4.1 Surrogate Models

Surrogate models are probabilistic approximations of the objective function that enable efficient exploration of the parameter space. While Gaussian Processes (GPs) are commonly used, other methods have gained popularity due to their flexibility and scalability. Here, I mention several types of surrogate models that are covered in the literature [**?**, **?**, **?**, **?**]:

4.4.2 Gaussian Processes (GPs)

Gaussian Processes (discussed thoroughly in **??**) provide a probabilistic representation of the objective function. For a given input $\mathbf{x}$, the GP provides a predictive distribution characterized by its mean $\mu(\mathbf{x})$ and variance $\sigma^2(\mathbf{x})$:

$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')) \tag{84}$$

where $k(\mathbf{x}, \mathbf{x}')$ is the kernel function that defines the covariance between any two points in the input space. GPs offer a natural way to model uncertainty and are particularly effective for low-dimensional problems.

4.4.3 Neural Networks

Neural Networks (NNs) have emerged as powerful surrogate models, especially for high-dimensional and large-scale optimization problems. They can capture complex, non-linear relationships in the data. In the context of BO, we often use:

- **Bayesian Neural Networks (BNNs)**: These assign a prior distribution to the network weights, allowing for uncertainty quantification. The predictive distribution for an input $\mathbf{x}$ is given by:

$$p(f(\mathbf{x})|\mathcal{D}) = \int p(f(\mathbf{x})|\mathbf{w})p(\mathbf{w}|\mathcal{D})d\mathbf{w} \tag{85}$$

  where $\mathbf{w}$ are the network weights and $\mathcal{D}$ is the observed data.

- **Ensembles of Neural Networks**: Multiple networks are trained on bootstrap samples of the data. The predictive mean and variance are estimated from the ensemble predictions:

$$\mu(\mathbf{x}) = \frac{1}{M}\sum_{i=1}^{M} f_i(\mathbf{x}), \quad \sigma^2(\mathbf{x}) = \frac{1}{M}\sum_{i=1}^{M}(f_i(\mathbf{x}) - \mu(\mathbf{x}))^2 \tag{86}$$

  where M is the number of ensemble members and $f_i(\mathbf{x})$ is the prediction of the i-th network.

4.4.4 Random Forests

Random Forests offer another alternative for surrogate modeling, particularly useful for categorical variables and mixed integer-continuous optimization problems. They provide uncertainty estimates through the variance of predictions across trees:

$$\mu(\mathbf{x}) = \frac{1}{T}\sum_{i=1}^{T} f_i(\mathbf{x}), \quad \sigma^2(\mathbf{x}) = \frac{1}{T}\sum_{i=1}^{T}(f_i(\mathbf{x}) - \mu(\mathbf{x}))^2 \tag{87}$$

where T is the number of trees and $f_i(\mathbf{x})$ is the prediction of the i-th tree.

4.4.5 Gaussian Process and Neural Network Hybrids

Recent research has explored hybrid models that combine the strengths of GPs and NNs. For instance, Deep Kernel Learning uses a neural network to learn features that are then fed into a GP:

$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\phi(\mathbf{x})), k(\phi(\mathbf{x}), \phi(\mathbf{x}'))) \tag{88}$$

where $\phi(\cdot)$ is a neural network feature extractor.

The choice of surrogate model depends on factors such as the dimensionality of the problem, the presence of categorical variables, the available computational resources, and the specific characteristics of the objective function. Each model offers different trade-offs between expressiveness, uncertainty quantification, and computational efficiency.

4.5 Exploration and Exploitation

A core concept in machine learning (specifically in decision making under uncertainty) is the trade-off between two competing strategies:

- **Exploration**: Selecting points in regions where the model uncertainty is high to improve the model's accuracy and coverage of the search space.

- **Exploitation**: Selecting points where the model predicts high values of the objective function to rapidly converge on the optimum.

Excessive exploration can lead to unnecessary evaluations in suboptimal areas, while over-exploitation might result in premature convergence to local optima, potentially missing the global optimum. Decision making on how to balance these two strategies is commonly referred to as the "Exploration/Exploitation Dilemma".

4.6 Multi-Armed Bandit Problem

The Exploration/Exploitation Dilemma in BO is closely related to the multi-armed bandit problem, a classic problem in decision theory and reinforcement learning. In the multi-armed bandit scenario, a gambler must decide which arm of several slot machines (bandits) to pull to maximize their cumulative reward. Each arm has an unknown probability distribution of rewards, and the gambler must balance the exploration of less-played arms and the exploitation of arms known to yield higher rewards.

Formally, the multi-armed bandit problem can be described as follows:

Let $\{a_1, ..., a_K\}$ be a set of K arms, each with an unknown reward distribution. At each time step t, the agent selects an arm a_t and receives a reward r_t. The objective is to maximize the cumulative reward over T time steps:

$$\max_{\{a_t\}_{t=1}^{T}} \mathbb{E}\left[\sum_{t=1}^{T} r_t\right] \tag{89}$$

This formulation closely mirrors the iterative nature of BO, where each "arm pull" corresponds to an evaluation of the objective function at a chosen point.

4.7 Acquisition Functions

Acquisition functions are the cornerstone of BO, guiding the selection of the next evaluation point by quantifying the exploration-exploitation trade-off. These functions utilize the predictive distribution provided by the surrogate model to assess the potential value of evaluating the objective function at a given point. Commonly used acquisition functions include:

4.7.1 Probability of Improvement (PI)

The Probability of Improvement acquisition function calculates the probability that a new point will yield an improvement over the current best observed value:

$$\alpha_{PI}(\mathbf{x}) = \Phi\left(\frac{\mu(\mathbf{x}) - f(\mathbf{x}^+) - \xi}{\sigma(\mathbf{x})}\right) \tag{90}$$

where $\mu(\mathbf{x})$ and $\sigma(\mathbf{x})$ are the mean and standard deviation predicted by the surrogate model at point $\mathbf{x}$, $f(\mathbf{x}^+)$ is the current best observed value, ξ is a small positive constant that controls the exploration-exploitation trade-off, and Φ is the cumulative distribution function of the standard normal distribution.

4.7.2 Expected Improvement (EI)

The Expected Improvement acquisition function calculates the expected value of the improvement over the current best observed value:

$$\alpha_{EI}(\mathbf{x}) = (\mu(\mathbf{x}) - f(\mathbf{x}^+) - \xi)\Phi\left(\frac{\mu(\mathbf{x}) - f(\mathbf{x}^+) - \xi}{\sigma(\mathbf{x})}\right) + \sigma(\mathbf{x})\phi\left(\frac{\mu(\mathbf{x}) - f(\mathbf{x}^+) - \xi}{\sigma(\mathbf{x})}\right) \tag{91}$$

where ϕ is the probability density function of the standard normal distribution.

4.7.3 Upper Confidence Bound (UCB)

The Upper Confidence Bound acquisition function provides a weighted sum of the predicted mean and standard deviation:

$$\alpha_{UCB}(\mathbf{x}) = \mu(\mathbf{x}) + \kappa\sigma(\mathbf{x}) \tag{92}$$

where κ is a parameter that controls the balance between exploration and exploitation. Higher κ values favor exploration by placing more emphasis on the uncertainty term.

4.7.4 Entropy Search and Predictive Entropy Search

These more advanced acquisition functions aim to maximize the information gain about the location of the global optimum. They are based on the principle of reducing the entropy of the posterior distribution over the location of the optimum.

For Entropy Search:

$$\alpha_{ES}(\mathbf{x}) = H(p(\mathbf{x}^*|D)) - \mathbb{E}_y \left[H(p(\mathbf{x}^*|D \cup \{(\mathbf{x}, y)\})) \right] \tag{93}$$

where $H(\cdot)$ denotes the differential entropy, $p(\mathbf{x}^*|D)$ is the posterior distribution over the location of the optimum given the current data D, and the expectation is taken over the predictive distribution of y at $\mathbf{x}$.

4.8 Optimization Process

The BO process can be summarized as follows:

1. Initialize the surrogate model with a small number of initial observations.
2. For $t = 1, 2, ..., T$:
   (a) Optimize the acquisition function to select the next point: $\mathbf{x}_t = \arg\max_{\mathbf{x}} \alpha(\mathbf{x})$
   (b) Evaluate the objective function at $\mathbf{x}_t$: $y_t = f(\mathbf{x}_t)$
   (c) Update the surrogate model with the new observation $(\mathbf{x}_t, y_t)$
3. Return the best observed solution

This iterative process continues until a stopping criterion is met, such as a maximum number of iterations or a satisfactory improvement threshold.

5 Deep Neural Networks

5.1 Multi-Layer Perceptron

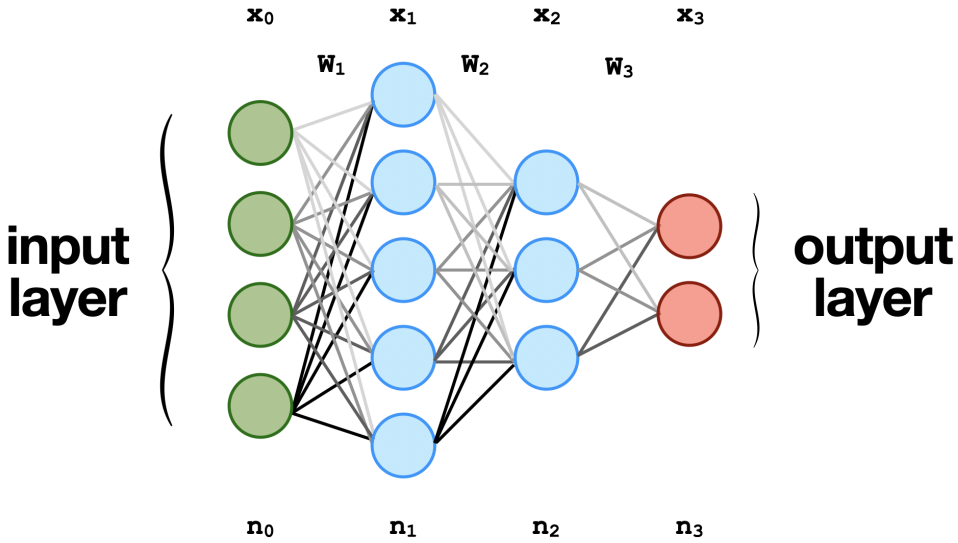


Figure 10: A typical visual representation of fully-connected 4-layer neural network or multi-layer perception (MLP) where $\boldsymbol{x}_i$ is input vector to the next layer, $\boldsymbol{W}_i$ are te matrix weights, and n_i are the number of neurons in a layer.

Neural networks are a subset of machine learning algorithms that are designed to model the behavior of the human brain. They are composed of interconnected nodes or neurons, which are organized into layers and process information in a parallel and distributed manner. Neural networks have gained significant attention in recent years due to their ability to solve complex problems in various domains, including image and speech recognition, natural language processing, and predictive analytics. One of the key strengths of neural networks is their ability to approximate any function to arbitrary accuracy, given a sufficiently large number of neurons in the hidden layers. This property, known as universal approximation, has made neural networks an essential tool for data scientists and researchers seeking to develop intelligent systems capable of learning and adapting to changing environments. In this background section, we will provide a comprehensive overview of the fundamentals of neural networks, including their architecture, training algorithms, and applications.

Multi-layer perceptions (MLPs) are a class of feed-forward neural networks that consist of one or more hidden layers of fully connected neurons (see Figure 10). In an MLP, each neuron receives inputs from all the neurons in the previous layer and computes a weighted sum of these inputs with additional bias term b. The weighted sum is then passed through an activation function, which introduces non-linearity into the model. The most commonly used activation functions include `sigmoid`, `ReLU`, and `tanh`. The output of each neuron is then passed on to the neurons in the next layer until the output layer is reached, which produces the final output of the network. This sequence of operations is typically referred to as the *forward pass*. The neural network *learns* by iteratively updating itself using optimization methods that minimizes the error between output of the neural network and the output from the training data, also known as the *backward pass* or *backward propagation*.

Forward Pass

The computations performed by the neurons in an MLP can be mathematically represented as matrix-vector multiplication, where the weights of each neuron in a layer are represented as a row in a weight matrix. Consider the MLP represented in Figure 10. The first layer contains four input nodes, which can be represented as an input vector $\boldsymbol{x}_0 \in \mathcal{R}^{n_0+1}$. The additional dimension is a place holder for the bias term b. The non-linear mapping between each layer begins with a linear matrix-vector operation between the preceding layer and its corresponding weight matrix $W_1 \in \mathcal{R}^{n_1 \times n_0+1}$. The output of this matrix is passed element-wise through a non-linear activation function of the user's choice. The output of this operation has a dimension equal to the size of the subsequent layer. Explicitly, this operation is

$$\phi(\boldsymbol{W}_1\boldsymbol{x}_0) = \phi\Bigg(\begin{bmatrix} W^1_{0,0} & W^1_{0,1} & W^1_{0,2} & W^1_{0,3} & b_0 \\ W^1_{1,0} & W^1_{1,1} & W^1_{1,2} & W^1_{1,3} & b_1 \\ W^1_{2,0} & W^1_{2,1} & W^1_{2,2} & W^1_{2,3} & b_2 \\ W^1_{3,0} & W^1_{3,1} & W^1_{3,2} & W^1_{3,3} & b_3 \\ W^1_{4,0} & W^1_{4,1} & W^1_{4,2} & W^1_{4,3} & b_4 \end{bmatrix} \begin{bmatrix} x^0_0 \\ x^0_1 \\ x^0_2 \\ x^0_3 \\ 1 \end{bmatrix}\Bigg) = \boldsymbol{x}_1 \tag{94}$$

where $W^1_{i,j}$ represents the element in the $(i+1)$-th row and $(j+1)$-th column of the weight matrix W_1, using zero-indexing, the superscript 1 indicates that $\boldsymbol{W}$ is weight matrix mapping to the first layer, and ϕ represents the activation function of choice.

Iterating this non-linear mapping procedure through each subsequent layer in Figure 10 yields the following result

$$\psi\Bigg(\underbrace{\boldsymbol{W}_3\,\phi(\boldsymbol{W}_2\underbrace{\phi(\boldsymbol{W}_1\boldsymbol{x}_0)}_{\boldsymbol{x}_1})}_{\boldsymbol{x}_2}\Bigg) = \boldsymbol{x}_3. \tag{95}$$

where the outer activation function ψ is the soft max function used to produce a probability distribution over the possible output classes.

Backward Pass

When training a model, inputs from a training data set are passed through a neural network following the matrix-vector multiplications provided above and the error between the outputs of the neural network and the training data can mathematically defined as the norm of the differences between both outputs:

$$\mathcal{L} = \frac{1}{2}||\boldsymbol{x}_3 - \boldsymbol{t}||^2_2 \tag{96}$$

where $\boldsymbol{x}_3$ is the output of the neural network, $\boldsymbol{t}$ is the ground truth from the training data, and $\mathcal{L}$ is the *loss*. Although there are a variety of optimization methods (more methods are discussed in later in this chapter) for determining the weights, the widely accepted approach is a first order gradient descent method [**?**]:

$$\boldsymbol{W}_i^{n+1} = \underbrace{\boldsymbol{W}_i^n}_{\in\mathcal{R}^{n_i\times n_{i-1}+1}} - \underbrace{\alpha\nabla_{\boldsymbol{W}_i}\mathcal{L}}_{\in\mathcal{R}^{n_i\times n_{i-1}+1}} \tag{97}$$

where $i = 1,\ldots,m$ represents the total number of transitions between layers. For the four layer network in Figure 10, there are three transitions meaning back propagation of this network will required differentiating $\mathcal{L}$ with respect to three weight matrices, $\boldsymbol{W}_1$, $\boldsymbol{W}_2$, and $\boldsymbol{W}_3$.

Back propagation of a four-layer fully connected neural network

The loss function we want to minimize is the L_2 norm of the difference between the output of the network and the training data:

$$\mathcal{L} = \frac{1}{2}||\boldsymbol{x}_3 - \boldsymbol{t}||^2_2 = \frac{1}{2}(\boldsymbol{x}_3 - \boldsymbol{t})^\top(\boldsymbol{x}_3 - \boldsymbol{t}) \tag{98}$$

where the explicit equation for the neural network is:

$$\boldsymbol{x}_3 = \psi\Bigg(\boldsymbol{W}_3\phi\Big(\boldsymbol{W}_2\phi\big(\boldsymbol{W}_1\boldsymbol{x}_0\big)\Big)\Bigg). \tag{99}$$

To update the next guess matrix for $\boldsymbol{W}_i$, we need to compute the gradients $\nabla\mathcal{L} = [\frac{\partial\mathcal{L}}{\partial\boldsymbol{W}_1}, \frac{\partial\mathcal{L}}{\partial\boldsymbol{W}_2}, \frac{\partial\mathcal{L}}{\partial\boldsymbol{W}_3}]$. It is important to note that since we are dealing partial derivatives with respect to matrices and vectors, the gradient of the loss function is actually a vector of matrices. The derivatives are obtained by using the chain rule:

$$\frac{\partial\mathcal{L}}{\partial\boldsymbol{W}_1} = \frac{\partial\mathcal{L}}{\partial\boldsymbol{x}_3}\circ\frac{\partial\boldsymbol{x}_3}{\partial\boldsymbol{x}_2}\circ\frac{\partial\boldsymbol{x}_2}{\partial\boldsymbol{x}_1}\circ\frac{\partial\boldsymbol{x}_1}{\partial\boldsymbol{W}_1}\circ\boldsymbol{x}_0^\top \tag{100}$$

An explicit computation of backpropagation is shown below.

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_1} = (\boldsymbol{x}_3 - \boldsymbol{t}) \circ \psi'(\boldsymbol{W}_3 \boldsymbol{x}_2) \circ \boldsymbol{W}_3 \phi'(\boldsymbol{W}_2 \boldsymbol{x}_1) \circ \boldsymbol{W}_2 \phi'(\boldsymbol{W}_1 \boldsymbol{x}_0) \circ \boldsymbol{x}_0^\top \tag{101}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_1} = (\boldsymbol{x}_3 - \boldsymbol{t}) \circ \psi'(\boldsymbol{W}_3 \boldsymbol{x}_2) \circ \boldsymbol{W}_3 \phi'(\boldsymbol{W}_2 \boldsymbol{x}_1) \boldsymbol{x}_1^\top \tag{102}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_3} = \underbrace{\underbrace{(\boldsymbol{x}_3 - \boldsymbol{t})}_{\in \mathcal{R}^{n_3}} \circ \underbrace{\psi'(\boldsymbol{W}_3 \boldsymbol{x}_2)}_{\in \mathcal{R}^{n_3}}}_{\begin{bmatrix} v_1 \\ v_2 \end{bmatrix}} \underbrace{\boldsymbol{x}_2^\top}_{\in \mathcal{R}^{1 \times n_2 + 1}} \begin{bmatrix} x_2^{(1)} & x_2^{(2)} & x_2^{(3)} & 1 \end{bmatrix} \tag{103}$$

where $\circ$ is the Hadamard product, and the final operation for each partial derivative is an outer product.

Inference from neural networks refers to the process of using a trained model to make predictions or decisions on new input data. During inference, the input data is passed through the network, which performs the sequential matrix-vector operations with the weights $\boldsymbol{W}_i$ obtained from training on the input to generate an output. The output is then used to make a prediction or decision based on the task the network was trained to perform, such as image classification or natural language processing. Inference is an important step in using neural networks for real-world applications, as it allows the model to be applied to new data and provide useful insights or actions.

5.2 Automatic Differentiation

In machine learning, the successful training of complex neural networks and mathematical models hinges upon the seamless interplay between two fundamental processes: optimization, which fine-tunes model parameters to minimize or maximize a specific objective, and backpropagation, which efficiently computes gradients essential for guiding the optimization process. Central to the efficacy of these processes are automatic differentiation tools, which provide a systematic and computationally efficient way to compute gradients of complex functions. Automatic differentiation techniques have emerged as a cornerstone technology, enabling the training of deep learning architectures and other optimization-driven algorithms. This methods-focused exposition delves into the landscape of automatic differentiation, elucidating its pivotal role in the domains of optimization and backpropagation. We explore the underlying principles of automatic differentiation, encompassing both forward and reverse modes, while highlighting prominent automatic differentiation frameworks and their integration within optimization algorithms. By elucidating the symbiotic relationship between automatic differentiation, optimization, and backpropagation, this section aims to equip practitioners with a comprehensive understanding of the techniques that underpin the training of modern machine learning models.

As indicated in the preceding section, deep learning is predicated on the successive matrix-vector operations followed by a non-linear activation function. Back-propagation requires differentiating through the entire network and requires a computational approach to keep track of an arbitrary number of mathematical operations. The fundamental representation in automatic differentiation is the computation graph which serves as a structured visualization of the transformations and computations that occur within a model during its forward pass.

By analyzing the chain of operations encoded within the computational graph, automatic differentiation enables the extraction of derivatives, facilitating efficient gradient computation for optimization algorithms like stochastic gradient descent. This dynamic interplay between computational graphs and automatic differentiation forms the foundation upon which modern optimization-driven machine learning thrives, engendering the ability to train intricate models while efficiently updating their parameters through gradient-based techniques.

6 Deep Reinforcement Learning

The following are my personal notes from Sergey Levine's CS 285 course in Fall 2022.

6.1 Policy Gradients Introduction

The goal of this assignment is to experiment with policy gradient and its variants, including variance reduction tricks such as implementing reward-to-go and neural network baselines. From a high level, the basic policy gradient algorithm is

1. generate samples (i.e., run the policy)
2. fit a model/estimate the return $J(\theta) = \mathbb{E}[\sum_t r_t] \approx \frac{1}{N}\sum_{i=1}^{N}\sum_t r_{t,i}$
   $\rightarrow$ found in `calculate_q_values`
3. improve the policy (backprop) $\theta \leftarrow \theta + \alpha\nabla\theta J(\theta)$
   $\rightarrow$ found in `MLPPolicyPG.update()`

6.2 Foundations of the Policy Gradient

At the core of any reinforcement learning algorithm is the policy, or internal model of the environment. For deep RL, this policy is represented as a neural network model that maps states $\boldsymbol{s}_t$ and/or observations $\boldsymbol{o}_t$ to actions $\boldsymbol{a}_t$, where t is a particular time step in the trajectory. The policy $\pi_\theta(\boldsymbol{a}_t|\boldsymbol{o}_t)$ is represented as a distribution of possible actions, given a set of observations. This can easily be encoded in PyTorch as a neural network that returns a distribution, as shown in the example code below:

```
class GaussianPolicy(nn.Module):
    def __init__(self, input_size, output_size):
        super(GaussianPolicy, self).__init__()

        # Mean will be a function of the input; std will be fixed (learned) value
        self.mean_fc1 = nn.Linear(input_size, 32)
        self.mean_fc2 = nn.Linear(32, 32)
        self.mean_fc3 = nn.Linear(32, output_size)
        self.log_std =  nn.Parameter(torch.randn(output_size))

    def forward(self, x):
        mean = F.relu(self.mean_fc1(x))
        mean = F.relu(self.mean_fc2(mean))
        mean = self.mean_fc3(mean)

        return distributions.Normal(mean, self.log_std.exp())
```

The next core concept in Deep RL, is the objective function that trains this neural network. As the agent interacts with its environment, we seek to learn about how the agent is rewarded when performing certain actions while being driven to a reward. For now, let us assume that we have a solid objective function that will allow us to adequately quantify the performance of the agent, and that the agent receives a reward at every time step. We now need to incorporate this reward into our policy so that the action distribution of the agent is shifted to those that will return the greatest reward. Here we introduce the Deep RL learning objective function:

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta(\tau)}[r(\tau)] \approx \frac{1}{N}\sum_i\sum_t r(s_{i,t}, a_{i,t}). \tag{104}$$

More specifically, we seek a set of neural weight parameters θ that minimizes Equation 104. The inner term in this equation $r(\tau)$ represents summation of rewards throughout an an entire trajectory, other was known as a rollout in Deep RL.

$$r(\tau) = r(\{s_0, a_0\}, ..., \{s_{T-1}, a_{T-1}\}) = \sum_{t=0}^{T-1} r(s_t, a_t) \tag{105}$$

The policy $\pi_\theta(\tau)$ is trained over an entire rollout τ of length T and attempts to map the as trajectory distribution $p_\theta(\tau)$. The trajectory distribution is a probability distribution over a sequence of states and actions:

$$p_\theta(\tau) = p(\{s_0, a_0\}, ..., \{s_{T-1}, a_{T-1}\}) = p(s_0)\pi_\theta(a_0|s_0)\prod_{t=1}^{T-1} p(s_t|s_{t-1}, a_{t-1})\pi_\theta(a_t|s_t). \tag{106}$$

In this representation, it expanded as the chain rule of probability as the product of the initial state distribution with the product the probability distributions overall time steps of the policy probability $\pi_theta(a_t|s_t)$, times the transition probability $p(s_{t+1}|s_t, a_t)$. The core assumption behind this equation is that the process is Markovian (see

lec-4 slide 12). In model-free RL, like policy gradients, we do not know the transition probabilities. However, we can effectively sample from the transition probabilities by interacting with the environment.

In training our policy, we would like to predict how well the actions of the agent will perform in the environment by sampling from our trajectory distribution $p_\theta(\tau)$, and estimating the potential reward. More explicitly, we would like to find a set of neural network weight parameters θ that maximizes the total summation of expected rewards over the entire trajectory:

$$\theta^* = \arg\max_\theta \mathbb{E}_{\theta\sim p_\theta(\tau)}\left[\sum_t r(s_t, a_t)\right]. \tag{107}$$

As stated in the title of this section, the purpose of the policy gradient approach is to directly take the gradient of Equation 104:

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta(\tau)}[r(\tau)] \tag{108}$$

$$= \int \nabla_\theta \pi_\theta(\tau) r(\tau) d\tau \tag{109}$$

Evaluating gradient of the objective in this form requires some manipulation, and will require the use of a "convenient identity".

A convenient identity

This directly follow from the equation of the derivative of the logarithm:

$$p_\theta(\tau)\nabla_\theta \log p_\theta(\tau) = p_\theta(\tau)\frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} = \nabla_\theta p_\theta(\tau)$$

The gradient of our objective can now be expressed as:

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\tau\sim p_\theta(\tau)}[r(\tau)] \tag{110}$$

$$= \int p_\theta \nabla_\theta \log p_\theta(\tau) r(\tau) d\tau \tag{111}$$

$$= \mathbb{E}_{\tau\sim p_\theta(\tau)}\left[\nabla_\theta \log p_\theta(\tau) r(\tau)\right] \tag{112}$$

Since we still do not know explicitly the trajectory distribution $p_\theta(\tau)$, we need further simplify this form. Here we refer back to Equation 106 and take the log of both sides. This yields:

$$\log p_\theta(\tau) = \log p_\theta(s_1) + \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) + \log p(s_{t+1}|s_t, a_t). \tag{113}$$

Since the first and third terms do not depend on the θ, the gradient of this function only yields the second term. This means we can re-write our policy gradient in terms of the neural network, independent of the transition policies:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta(\tau)}\left[\nabla_\theta \log p_\theta(\tau) r(\tau)\right] \tag{114}$$

$$= \mathbb{E}_{\tau\sim\pi_\theta(\tau)}\left[\left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\right)\left(\sum_{t=1}^{T} r(s_t, a_t)\right)\right] \tag{115}$$

We need to make one more modification to get the basic form of our policy gradient. Recall from Equation 104 that the objective can be approximated as a double summation over all rollouts, and their subsequent time steps. This means that we can also approximate the gradient of the objective as well:

Policy gradient:

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t})\right)\left(\sum_{t=1}^{T} r(s_{i,t}, a_{i,t})\right) \tag{116}$$

Using this form of the policy gradient, we can now implement what is referred to as the REINFORCE algorithm.

while True:

1. sample $\{\tau^i\}$ from $\pi_\theta(a_t|s_t)$ (i.e., run the policy)
2. $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) \right) \left(\sum_{t=1}^{T} r(s_{i,t}, a_{i,t}) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

The summation of the rewards in Equation 116 is part of the green box in Figure 11 and is where we fit a model to estimate a return. The outer summation overall trajectories is processed in the orange box in Figure 11 and is where sample generation occurs. To improve the policy (blue box), θ is updated according to step 3 above.

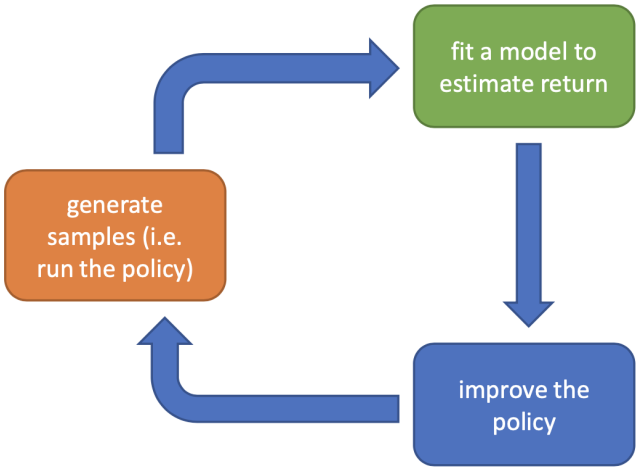


Figure 11: REINFORCE algorithm

The basic idea behind this algorithm is that we want the objective function to capture the probability of reward scaled by the reward from the last action. By iterating over and over within an environment, we begin to bias the policy to produce an action distribution that prioritizes distributions that yield the highest reward (see Figure 12).

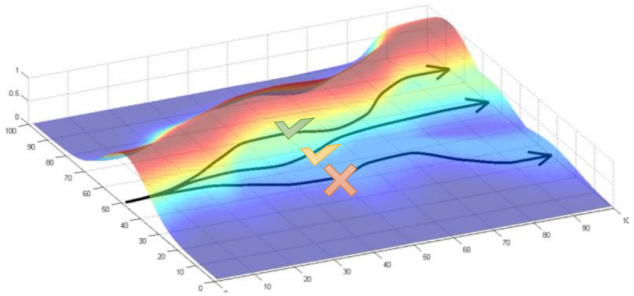


Figure 12: Learning distribution

6.3 Implementation of vanilla policy gradient with PyTorch

On assignment 2, we were required to implement several variations of a policy gradient. This required coding Equation 116 in several ways. Each method will be described below, but here I will describe at a high level how each term of the policy gradient in the current form can be computed with PyTorch.

We seek to encode a PyTorch loss function that encapsulates all of terms in Equation 116. We will first examine the first inner summation over the log probabilities:

$$\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}).$$

Since we have a neural network (policy $\pi_\theta(a_t|s_t)$) that outputs a distribution of potential actions from inputted observations (see https://pytorch.org/docs/stable/distributions.html):

```
loss = -action_distribution.log_prob(action)
```

The second summation of the rewards over an entire rollout is

$$\sum_{t=1}^{T} r(s_{i,t}, a_{i,t}) \tag{117}$$

and is computed in the assignment as

```
# a function that returns the summation of rewards over an entire rollout
return = lambda rewards: (np.ones(len(rewards)) * sum(rewards)).tolist()
```

Assume for now that the advantage is equivalent to the reward. We can add this to the loss function and implement the REINFORCE algorithm as:

```
# step 1: generate samples
action_distribution = self.forward(observations)
action = action_distribution.sample()  # in hw performed upstream from MLP_policy
next_state, reward = env.step(action)  # in hw performed upstream from MLP_policy

# step 2: fit a model to estimate the return
loss = -action_distribution.log_prob(action) * reward

# step 3: improve policy (backprop)
loss.backward()
```

6.4 Variance reduction: reward-to-go

In this assignment, we implemented the policy gradient (Equation 116) in a variety of ways. We discussed a number of issues that affected the vanilla implementation, and specifically the variance of the action distribution provided by the policy (see Figure 12 for a visual understanding). The additional methods that we sought to implement mainly focused on reducing this variance.

One way to reduce the variance of the policy gradient is to exploit **causality**: the notion that the policy cannot affect rewards in the past. This yields the following modified objective, where the sum of rewards here does not include the rewards achieved prior to the time step at which the policy is being queried. This sum of rewards is a sample estimate of the Q function, and is referred to as the "reward-to-go."

Policy gradient with reward-to-go:

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t})\right)\left(\sum_{t'=t}^{T} r(s_{i,t'}, a_{i,t'})\right) \tag{118}$$

Since the only thing that has changed between Equations 116 and 118 is the last term, the only thing that is changed in the PyTorch implementations is the computation of the reward:

Computing reward-to-go:

```
# computing the summation of rewards over an entire rollout, with reward-to-go
def return(rewards):
    cumsum_return = np.ones(len(rewards)) * sum(rewards)
    for i in range(1, len(rewards)):
        # subtract off the previous time step's reward
        cumsum_return[i] -= rewards[i - 1]
    return cumsum_return.tolist()
```

6.5 Variance reduction: discounting

Multiplying a discount factor γ to the rewards can be interpreted as encouraging the agent to focus more on the rewards that are closer in time, and less on the rewards that are further in the future. This can also be thought of as a means for reducing variance (because there is more variance possible when considering futures that are further into the future). We saw in lecture that the discount factor can be incorporated in two ways, as shown below.

The first way applies the discount on the rewards from full trajectory:

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t})\right)\left(\sum_{t'=1}^{T} \gamma^{t'-1} r(s_{i,t'}, a_{i,t'})\right) \tag{119}$$

which can be applied in PyTorch by modifying the reward implementations from above:

Computing the discounted reward:

```
# a function that returns the summation of discounted rewards over an entire rollout
def _discounted_return(rewards: list) -> list:
    total_discounted_return = sum([self.gamma ** t * r for t, r in enumerate(rewards)])
    list_of_discounted_returns = np.ones(len(rewards)) * total_discounted_return

    return list_of_discounted_returns.tolist()
```

and the second way applies the discount on the "reward-to-go:"

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t})\left(\sum_{t'=1}^{T} \gamma^{t'-1} r(s_{i,t'}, a_{i,t'})\right). \tag{120}$$

Computing the discounted reward-to-go:

```
# a function that returns the summation of discounted reward-to-gos
# over an entire rollout
def _discounted_cumsum(rewards: list) -> list:
        all_discounted_cumsums = []

        # for loop over steps (t) of the given rollout
        for start_time_index in range(len(rewards)):

            # 1) create an array of indices (t'): goes from t to T-1
            indices = np.arange(start_time_index, len(rewards))

            # 2) create an array where the entry at each index (t') is gamma^(t'-t)
            discounts = np.power(self.gamma, indices - start_time_index)

            # 3) create a list where the entry at each index (t')
            #    is gamma^(t'-t) * r_{t'}
            discounted_rtg = rewards[indices] * discounts

            # 4) calculate a scalar: sum_{t'=t}^{T-1} gamma^(t'-t) * r_{t'}
            sum_discounted_rtg = np.sum(discounted_rtg)

            # appending each of these calculated sums into the list to return
            all_discounted_cumsums.append(sum_discounted_rtg)
        list_of_discounted_cumsums = np.array(all_discounted_cumsums)
        return list_of_discounted_cumsums.tolist()
```

6.6 Variance reduction: baseline

Another variance reduction method is to subtract a baseline (utilizing the "convenient identity", and that is a constant with respect to τ) from the sum of rewards:

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}\big[r(\tau) - b\big] = \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta} \nabla_\theta \log p_\theta(\tau)\big[r(\tau) - b\big]. \tag{121}$$

This leaves the policy gradient unbiased (in expectation, not variance) because:

$$\begin{aligned}
\nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta(\tau)}\big[\nabla_\theta \log p_\theta(\tau)\cdot b\big] &= 0 && (122)\\
&= \int p_\theta(\tau)\nabla_\theta p_\theta(\tau) b d\tau && (123)\\
&= \int \nabla_\theta p_\theta(\tau) b d\tau && (124)\\
&= b\nabla_\theta \int p_\theta \int p_\theta(\tau) d\tau && (125)\\
&= b\nabla_\theta 1 && (126)\\
&= 0 && (127)
\end{aligned}$$

In this assignment, we will implement a value function V^π_ϕ which acts as a state-dependent baseline. This value function will be trained to approximate the sum of future rewards starting from a particular state:

$$V^\pi_\phi \approx \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}\big[r(s_{t'}, a_{t'})|s_t\big], \tag{128}$$

so the approximate policy gradient now looks like this:

Policy gradient with baseline:

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(s_{i,t}|a_{i,t})\Big(\big(\sum t' = t^{T-1}\gamma^{t'-t}r(s_{i,t'},a_{i,t'})\big) - V_\phi^\pi(s_{i,t})\Big) \tag{129}$$

The right-most hand term $\big(\big(\sum t' = t^{T-1}\gamma^{t'-t}r(s_{i,t'},a_{i,t'})\big) - V_\phi^\pi(s_{i,t})\big)$ is practically computed separately. First, a discount is applied to reward, and then the reward, or reward-to-go, is passed on as the "Q-value".

Estimating Q values:

```
def calculate_q_vals(self, rewards_list):
    if not self.reward_to_go:
        discounted_return = [self._discounted_return(reward)
                             for reward in rewards_list]
        q_values = np.concatenate(discounted_return)
    else:
        q_values = np.concatenate([self._discounted_cumsum(i) for i in rewards_list])
    return q_values
```

The value function in Equation 128 can be "learned" by mapping the observed states to the rewards, and returning the summation of the expectations. For this estimate, it is important to normalize the observations prior to training the multi-layer perceptron. An implementation is listed below:

Learning the value function:

```
# setting up the MLP in  MLPPolicy
if nn_baseline:
    self.baseline = ptu.build_mlp(
        input_size=self.ob_dim,
        output_size=1,
        n_layers=self.n_layers,
        size=self.size,
    )
    self.baseline.to(ptu.device)
    self.baseline_optimizer = optim.Adam(
        self.baseline.parameters(),
        self.learning_rate,
    )

...

# within the update function
if self.nn_baseline:
    # normalize rewards
    targets = ptu.from_numpy((q_values - np.mean(q_values)) / (np.std(q_values) + 1e-8))
    baseline_prediction = self.baseline(observations).squeeze()
    baseline_loss = self.baseline_loss(baseline_prediction, targets)

    self.baseline_optimizer.zero_grad()
    baseline_loss.backward()
    self.baseline_optimizer.step()

    train_log = {'Baseline Loss': ptu.to_numpy(baseline_loss), }
```

In our implementation, the actual estimation of `values` from `nn_baseline()` is computed in `PGAgent` while estimating the *advantage*. *Recall* that the advantage is defined as the difference between the `q_values - values`.

Estimating advantage:

```
def estimate_advantage(self,
                       obs: np.ndarray,
                       rews_list: np.ndarray,
                       q_values: np.ndarray,
                       terminals: np.ndarray):
    if self.nn_baseline:
        values_unnormalized = self.actor.run_baseline_prediction(obs)
        assert values_unnormalized.ndim == q_values.ndim

        # raise values
        values = values_unnormalized * np.std(q_values) + np.mean(q_values)
        advantages = q_values - values
    else:
        advantages = q_values.copy()

    if self.standardize_advantages:
        advantages = (advantages - np.mean(advantages)) / (np.std(advantages) + 1e-8)

    return advantages
```

6.7 Variance reduction: generalized advantage estimation:

The quantity $\left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{t'}, a_{t'})\right) - V_\phi^\pi(s_t)$ from the previous policy gradient expression (removing the i index for clarity) can be interpreted as an estimate of the advantage function:

$$A^\pi(s_t, a_t) = \left(\sum t' = t^{T-1} \gamma^{t'-t} r(s_{i,t'}, a_{i,t'})\right) - V_\phi^\pi(s_{i,t}) \tag{130}$$

$$= Q^\pi(s_t, a_t) - V^\pi(s_t) \tag{131}$$

where $Q^\pi(s_t, a_t)$ is estimated using Monte Carlo returns and $V^\pi(s_t)$ is estimating using the learned value function V_ϕ^π. We can further reduce variance by also using V_ϕ^π in place of the Monte Carlo returns to estimate the advantage function as

$$A^\pi(s_t, a_t) \approx \delta_t = r(s_t, a_t) + \gamma V_\phi^\pi(s_{t+1}) - V_\phi^\pi(s_t), \tag{132}$$

with the edge case $\delta_{T-1} = r(s_{T-1}, a_{T-1}) - V_\phi^\pi(s_{T-1})$. However, this comes at the cost of introducing bias to our policy gradient estimate, due to modeling errors in V_ϕ^π. We can instead use a combination of n-step Monte Carlo returns and V_ϕ^π to estimate the advantage function as:

$$A_n^\pi(s_t, a_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(s_{t'}, a_{t'}) + \gamma^n V_\phi^\pi(s_{t+n+1}) - V_\phi^\pi(s_t). \tag{133}$$

Increasing n incorporates the Monte Carlo returns more heavily in the advantage estimate, which lowers bias and increases variance, while decreasing n does the opposite. Note that $n = T - t - 1$ recovers the unbiased but higher variance Monte Carlo advantage estimate used in Equation 129, while $n = 0$ recovers the lower variance but higher bias advantage estimate δt.

We can combine multiple n-step advantage estimates as an exponentially weighted sum, which is known as the generalized advantage estimator (GAE). Let $\lambda \in [0, 1]$. Then we define:

$$A_{GAE}^\pi(s_t, a_t) = \frac{1 - \lambda^{T-t-1}}{1 - \lambda} \sum_{n=1}^{T-t-1} \lambda^{n-1} A_n^\pi(s_t, a_t), \tag{134}$$

where $\frac{1-\lambda^{T-t-1}}{1-\lambda}$ is a normalizing constant. Note that the higher λ exmphasizes advantage estimates with higher values of n, and a lower λ does the opposite. In the infinite horizon case ($T = \infty$), we can show that

$$A^{\pi}_{GAE}(s_t, a_t) = \sum_{t'=t}^{T-1} (\gamma\lambda)^{t'-t} \delta_{t'}, \tag{135}$$

which serves as a way we can efficiently implement the generalized advantage estimator since we can recursively compute:

$$A^{\pi}_{GAE}(s_t, a_t) = \delta_t + \gamma\lambda A^{\pi}_{GAE}(s_{t+1}, a_{t+1}) \tag{136}$$

Computationally, we can implement this recursion as:

Computing the generalized advantage estimation (GAE) → `estimate_advantage()`:

```
if self.nn_baseline:
        values_unnormalized = self.actor.run_baseline_prediction(obs)
        assert values_unnormalized.ndim == q_values.ndim

        values = values_unnormalized * np.std(q_values) + np.mean(q_values)

        if self.gae_lambda is not None:
            values = np.append(values, [0])

            rews = np.concatenate(rews_list)

            batch_size = obs.shape[0]
            advantages = np.zeros(batch_size + 1)

            for i in reversed(range(batch_size)):
                if terminals[i] == 1:
                    advantages[i] = rews[i] - values[i]
                else:
                    advantages[i] = rews[i] + self.gamma * values[i + 1] - values[i]
                    # equation 33 above
                    advantages[i] += self.gamma * self.gae_lambda * advantages[i + 1]

            advantages = advantages[:-1]

        else:
            advantages = q_values - values

    else:
        advantages = q_values.copy()
```

6.8 Actor-critic algorithms

To improve on this framework, we would like to make some modifications to the format above. Rather than strictly summing up the rewards, we will fit a separate neural network to estimate either Q^π, V^π, A^π. This means that we will have two neural networks within the policy gradient algorithm (Figure 11), specifically in the green box "fit a model to the estimate return":

- a policy network (actor that return actions)
- a value function network (critic that criticizes or baselines)

The reason why we choose to approximate the value function is because we can the value function in the next state can be written as:

$$V^\pi(s_{t+1}) = r(s_t, a_t) + \sum_{t'=t+1}^{T} E_{\pi_\theta}\big[r(s_{t'}|s_t, a_t)\big]$$

additionally, since we know the current reward, we can separate it out from the expectation of the quality function:

$$Q^\pi(s_t, a_t) = \sum_{t'=t}^{T} E_{\pi_\theta}\big[r(s_{t'}, a_{t'}|s_t, a_t)\big] \quad \rightarrow \quad r(s_t, a_t) + \sum_{t'=t+1}^{T} E_{\pi_\theta}\big[r(s_{t'}, a_{t'})|s_t, a_t\big]$$

where all $s_{t'}$ after time step t are random variables. Now we can see that the second component of the quality function is just the value function at the next time step. We know that the expectation of all rewards in the future is perfectly summarized with by the expected value of $V^\pi(s_{t+1})$, where $s_{t+1} \sim p(s_{t+1}|s_t, a_t)$:

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + E_{s_{t+1}\sim p(s_{t+1}|s_t,a_t)}\big[V^\pi(s_{t+1})\big]. \tag{137}$$

With this new approximation to the quality function, we can arrive at advantage function for the actor-critic algorithms:

$$A^\pi(s_t, a_t) \approx r(s_t, a_t) + V^\pi(s_{t+1}) - V^\pi(s_t) \tag{138}$$

which means we can express the advantage and quality functions in terms of just the value function.

The actor critic algorithm
`while not done do:`

1. sample $\{s_i, a_i\}$ from $\pi_\theta(a|s)$ (run the policy in simulation or on a robot)
2. fit $\hat{V}^\pi_\phi$ to the sampled reward sums
3. evaluate $\hat{A}^\pi(s_i, a_i) = r(s_i, a_i) + \hat{V}^\pi_\phi(s'_i) - \hat{V}^\pi_\phi(s_i)$
4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(a_i|s_i)|s_{i_t})\hat{A}^\pi(s_i, a_i)$
5. $\theta \leftarrow \theta + \alpha\nabla_\theta J(\theta)$
6. return to 1

6.9 Q-learning

Recall from Equation 128 (also listed below as a reminder) is the expected reward given being in some state s_t provided that the best action is taken $r(s_{t'}, a_{t'})$. In other words, the value function attempts to quantify the value of being in particular state, *assuming that the agent takes the best action.*

$$V^\pi_\phi \approx \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}\big[r(s_{t'}, a_{t'})|s_t\big],$$

Q-learning attempts generalize the value function by quantifying the *quality* of being in a particular state s_t, for all possible actions a_t. The quality of being in a state now is my expected current reward in addition to all of the future rewards for being in the next state $s_{t'}$. We can write this as a sum of probabilities (since this is a Markov-Decision Process) over all possible next states $s_{t'}$

$$Q_\phi^\pi \approx \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}\left[r(s_{t'}, s_t, a_t) + \gamma V_\phi^\pi(s_{t'})\right] = \sum_{s_{t'}} P(s_{t'}|s_t, a_t) \cdot \left(r(s_{t'}, s_t, a_t) + \gamma V_\phi^\pi(s_{t'})\right) \tag{139}$$

In Equation 139, the value function that is being used can be computed as

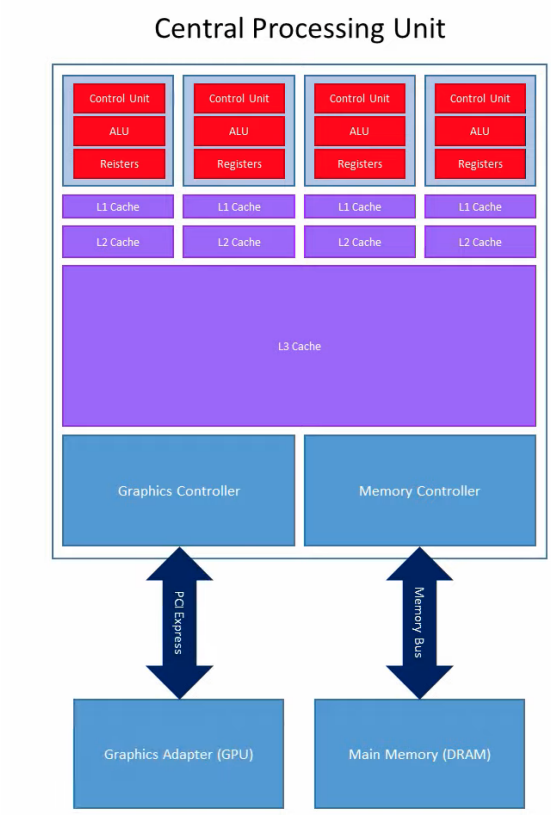
$$V_\phi^\pi(s) = \arg\max_{a_t} Q(s_t, a_t) \tag{140}$$

7 Computing

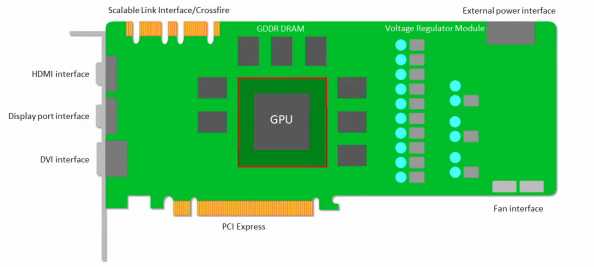
7.1 CPU + GPU Architecture

The CPU is responsible for initiating computation on the GPU and for transferring data to and from the device (see Figure 13a). The GPU is a specialized accelerator designed to handle operations that can be massively parallelized (see Figure 13b). This is accomplished via a high-speed bus, typically PCIe (Peripheral Component Interconnect Express). The GPU has its own high-bandwidth memory, referred to as HBM (High Bandwidth Memory) or GDDR, depending on the architecture. For the NVIDIA A100, this is HBM2e, typically 40GB or 80GB in size. This memory is optimized for massive bandwidth (TB/s range) rather than the low latency optimized for CPU DRAM.

External connections include voltage regulator modules for power modulation, thermal management systems, and the PCIe interface. Crucially for HPC, high-performance GPUs like the A100 utilize NVLink, a high-speed interconnect that allows multiple GPUs to communicate directly with each other at speeds significantly faster than PCIe, bypassing the CPU for peer-to-peer transfers.



(a) Typical architecture of a multi-core CPU. Each core contains 1) control unit, 2) arithmetic-logic units (ALU), 3) vector registers. Memory buses transfer information between the cache (SRAM) and main memory (DRAM).



(b) High-level representation of GPU hardware. Key inputs include the PCIe bus (Host-to-Device) and NVLink (Device-to-Device), alongside power and thermal regulation.

Figure 13: Relationship between CPU and GPU.

At the center of the hardware is the GPU processor itself. A modern GPU is composed of many independent execution units called Streaming Multiprocessors (SMs). **The SM is the equivalent of a CPU Core regarding architectural independence.** Each SM maintains the state for typically up to 2048 threads (managed in groups called Warps). The threads executing on a single SM can communicate through a fast, programmable on-chip memory called Shared Memory and synchronize using lightweight barriers.

One level below the SM is the **Processing Element (PE)**. In architecture literature, this is often called a "Lane" or "ALU" (Arithmetic Logic Unit). In NVIDIA marketing terminology, these are referred to as "Cores" (e.g., CUDA Cores, Tensor Cores). Each SM has its own on chip private memory (SRAM) that is partitioned into L1 Data Cache and Shared Memory. 'cudaFuncSetAttribute' is CUDA setting to "carveout" the L1/Shared Memory

ratio manually. For A100s, each SM has 192kb of unified L1/shared memory. There is a fundamental trade off between use of '__shared__' since the L1 cache will fundamentally have less capacity

It is crucial to distinguish between the **Thread** and the **PE**:

- **Thread (Software):** A logical unit of execution with its own register state and instruction pointer. An A100 SM can maintain the state for up to 2048 resident threads simultaneously.
- **PE (Hardware):** The physical execution lane that performs the arithmetic. An A100 SM has 64 FP32 PEs.

The SM's Warp Scheduler dynamically assigns the physical PEs to execute instructions for the resident threads, allowing the hardware to hide memory latency by swapping between threads instantly.

7.2 GPU Hardware to Software Abstraction

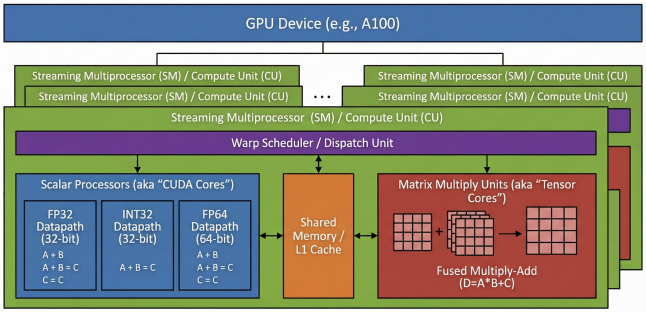
When learning about GPU programming for the first time, I experienced confusion in mapping software abstraction concepts to the hardware reality. This hierarchy is how I mentally map the two together:

- **Thread → CUDA core lane**: A single thread of execution. Mapped to a single lane on a CUDA Core.
- **Warp → Warp Scheduler**: A collection of 32 threads that execute the same instruction in lockstep (SIMT).
- **Thread Block → Streaming Multiprocessor (SM)**: A user-defined group of threads that can share memory and synchronize. A block is assigned to a single SM and resides there until completion.
- **Grid → Device (GPU)**: The entire kernel launch, comprising many thread blocks. The GPU's hardware work distributor assigns blocks to available SMs across the entire device.

Table 1: Hardware to Software Abstraction

Software Abstraction	Hardware Execution Unit
Thread	CUDA Core / SIMT Lane
Warp (32 Threads)	Warp Scheduler / SIMT Unit
Thread Block	Streaming Multiprocessor (SM)
Grid (Kernel Launch)	Device (Entire GPU)

(a) A visual representation of the relationship between hardware and software terminology. The red represents the software abstractions and the green represents the hardware terminology.



(b) Alternative datapaths available to Tensor Cores and CUDA cores (sketched by hand and then autogenerated with NanoBanana).

Figure 14: Visualizations of GPU software abstraction mapping (a) and internal hardware hierarchy (b).

The number of SMs in a Nvidia GPU and the number of PEs in each SM can vary depending on the specific GPU model and architecture. Nvidia groups 32 PEs as a **warp** of threads or **single-instruction multiple-threads** (SIMT). For Perlmutter A100s, there are 108 SMs each containing 64 warps, meaning that there are about 200,000+ ways of parallelism (108*64*32) per GPU.

A CUDA core differs from a Tensor core in that CUDA cores are general-purpose processing units that can perform arithmetic and logic operations on floating-point and integer data types. CUDA cores are used for a wide range of tasks, including graphics rendering, scientific simulations, and data processing.

Tensor cores, on the other hand, are specialized hardware units that are designed specifically for performing matrix operations, which are a fundamental building block of many machine learning algorithms. Tensor cores are capable of performing mixed-precision matrix multiplication and accumulation operations with high throughput and low latency, which makes them well-suited for accelerating deep learning workloads.

7.3 Topics in CUDA Programming

Memory Coalescing

Within CUDA kernel, global memory access (when a thread calls data from memory) occurs via "transactions" in 32 byte sectors, within a 128-byte cache line. A memory accesses to VRAM/HBM must be optimized given the transaction size. If data requested from each thread is not contiguous in memory, the system must request adjacent memory slots that do not benefit the computation. For example, the following kernel access two memory slots `in` and `out`:

```
#include <mma.h>
using namespace nvcuda;

__global__ void stride_kernel(float *in, float *out, int stride) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;

    // each thread id reads from continugous block of memory
    float val = in[tid];

    // each thread id access a slot in memory in strides
    // causing a major slowdown
    out[tid * stride] = val;
}
```

Listing 1: Memory coalescing

Since accesses to `in` are contiguous in memory, the GPU memory controller can grab 32-byte sectors along the entire cache line with optimal efficiency. However, when accessing `out`, the memory slot jumps by a multiple of `stride`, which will force bring whatever data is in between into the cache from VRAM, dropping efficiency by that `stride` multiple. If `stride=2`, there will then be a 8-byte gap (an empty float position). Now, the entire span of the data is doubled (256 bytes) across a minimum of two cache lines, thus reducing the bus utilization by 50%. If the `stride` = 32, now each float is actually taking up 128 bytes (an entire cache line), slowing down memory access by 32.

Shared Memory Bank Conflicts: Column Access Logic

Each SM has Shared Memory that is partitioned into 32 independent modules called **banks**. Each bank has a word width of 4 bytes (32 bits), where successive 4-byte words are assigned to successive banks following:

$$\text{Bank ID} = (\text{Address} \div \text{Word Size}) \% 32$$

.

Every address in shared memory (in increments of 4 bytes) has a fixed determistic mapping to a specific bank on its address. This mapping never changes. For 4-byte elements (like `float` or `int`):

$$\text{Bank number} = \frac{\text{Byte Address}}{4} \pmod{32}$$

or equivalently the element index:

$$\text{Bank number} = \text{Element Index} \pmod{32}$$

Bank conflicts can happen when multiple warps attempt to access the same bank. For example, in a 32×32 matrix of 4-byte floats stored in shared memory (row-major), thread access to columns and rows will result in very different performance. For example, the 32 floats in the first row will map nicely to each bank: `mat[row + tid][col]`

→ `bank_0-31`. However, accessing a column: `mat[0][0]` → `bank_0` , `mat[1][0]` → `bank_0`, etc., will continue to attempt accesses at bank 0.

Padding can break misalignment:

```
#define N 32
#define PAD 1
__shared__ float tile[N][N+PAD]; // add extra column
float value = tile[tid][0];
```

Listing 2: Padding

Now the rows are offset:

- **Row 0:** starts at address 0 (Bank 0)
- **Row 1:** starts at address $33 \times 4 = 132$ bytes (Bank 1)
- **Row 2:** starts at address $33 \times 8 = 264$ bytes (Bank 2)

Now each thread accesses a different bank with now conflicts.
To calculate the padding, for an array with stride s:

$$\text{Padding} = gcd(s, 32) - 1 \pmod{32}$$

where gcd is the greatest common divisor.

Perlmutter GPU specifications

NERSC's Perlmutter super computer utilizes 4 NVIDIA Ampere A100 GPUs which are powerful computing units designed for massively-parallelized computing and artificial intelligence applications. Here are the specifications of the A100 GPU:

Query	**Result**	**Command**
GPU	NVIDIA A100-SXM4-80GB	`nvidia-smi --query-gpu=name --format=csv`
SM's	108	see number of streaming multiprocessors (SMs)
max threads per SM	2048	see max number of threads per SM
max threads per GPU	221184 (=108×2048)	multiply the two above
maximum x-dim of thread blocks grid	$2^{31} - 1$	see Table 15 here
cores	≥6,912	see total number of cores
GPU Memory	81920 MiB (∼86GB)	`nvidia-smi --query-gpu=memory.total --format=csv`

Table 2: Specifications of the NVIDIA Ampere A100 GPU and corresponding commands to check.

number of streaming multiprocessors (SMs)

```
    int deviceCount;
    cudaGetDeviceCount(&deviceCount);

    for (int i = 0; i < deviceCount; ++i) {
        cudaDeviceProp deviceProp;
        cudaGetDeviceProperties(&deviceProp, i);
        std::cout << "Device " << i << " has " << deviceProp.multiProcessorCount << " SMs\n";
    }
```

max number of threads per SM

```
    int deviceCount;
    cudaGetDeviceCount(&deviceCount);

    for (int i = 0; i < deviceCount; ++i) {
        cudaDeviceProp deviceProp;
        cudaGetDeviceProperties(&deviceProp, i);
        std::cout << "Device " << i << " has maximum threads per SM: " << deviceProp.
    maxThreadsPerMultiProcessor << "\n";
    }
```

total number of cores

The best way to see the cores within a single streaming multiprocessor (SM) on an NVIDIA GPU is to look up the major revision and the minor revision of the GPU with the following code:

```
    int deviceCount;
    cudaGetDeviceCount(&deviceCount);

    for (int i = 0; i < deviceCount; ++i) {
        cudaDeviceProp deviceProp;
        cudaGetDeviceProperties(&deviceProp, i);
        std::cout << "Device " << i << ":\n";
        std::cout << "  deviceProp.major: " << deviceProp.major << "\n";
        std::cout << "  deviceProp.minor: " << deviceProp.minor << "\n";
    }
```

Running this code on an NVIDIA A100 revealed that it has a major revision of `8` and a minor revision of `0`. Now, we just need look up the specs for NVIDIA `8.0` GPUs here and find that a single SM on a NVIDIA `8.0` GPU consists of:

- 64 FP32 cores for single-precision arithmetic operations
- 32 FP64 cores for double-precision arithmetic operations
- 64 INT32 cores for integer math
- 4 mixed-precision Third-Generation Tensor Cores supporting half-precision (fp16), __nv_bfloat16, tf32, sub-byte and double precision (fp64) matrix arithmetic
- 16 special function units for single-precision floating-point transcendental functions
- 4 warp schedulers

We can run performance analysis on any parallelized code by using Nvidia's software Nsight Systems (download Nsight Systems 2023.1.1 (macOS Host)).

nsys profile ./gpu -n 1000000 -s 1